

Documentación Técnica Exhaustiva de Métodos y Diagramas de Flujo de Procesos

Manual de ingeniería inversa y arquitectura del sistema de compilación **NovaNyx 1.4.2** (Proyecto *Interfaz Léxico*). Incluye descripción textual detallada, contratos de signature y diagramación de flujo vectorial (SVG) para la totalidad de los métodos implementados.

9

MÓDULOS / CLASES

88

MÉTODOS DOCUMENTADOS

88

DIAGRAMAS VECTORIALES

100%

COBERTURA DE CÓDIGO

Proyecto: Interfaz Lexico_LA2 · Lenguaje NovaNyx 1.4.2

Plataforma: C# .NET / Windows Forms / Microsoft SQL Server

Fecha de Generación: 17 de September de 2026

Estado: Documento Completo y Verificado

Índice General de Módulos y Métodos

Archivo<Tipo>

(10 métodos)

Archivo(string strArchivo)
Crear()
~Archivo()
HacerModoEscritura()
HacerModoLectura()
AgregarObjeto(Tipo Objeto)
LeerObjeto()
CerrarArchivo()
EliminarArchivo()
FinArchivo [Propiedad get]

ClaseNodo<Tipo>

(2 métodos)

~ClaseNodo()
ObjetoConDatos, Anterior, Siguiente [Propiedades]

Identificador

(2 métodos)

Equals(Identificador other)
CompareTo(Identificador other)

NodoJerarquia

(2 métodos)

NodoJerarquia() y Sobrecarga con Parámetros
ToTreeNode()

ClaseListaSimpleOrdenada<Tipo>

(7 métodos)

ClaseListaSimpleOrdenada() [Constructor]
GetEnumerator()
Insertar(Tipo Objeto)
Eliminar(Tipo Objeto)
Buscar(Tipo Objeto)
Vaciar() y ~ClaseListaSimpleOrdenada()
Contar [Propiedad get]

AnalizadorSintactico

(19 métodos)

AnalizadorSintactico(...) [Constructor]
EsDelimitador(TokenSintactico tok)
MatchDelimitador()
ExigirDelimitador(string contexto = "")
SincronizarHastaDelimitador()
ParsearPrograma()
ParsearInstrucciones(params string[] stopTokensAdicionales)
EsFinDeBloque(string[]? stopTokensAdicionales = null)
ParsearInstruccion()
ParsearCondicion()
ParsearCondicionOExpresion()
ParsearExpresion()
ParsearTermino()
ParsearPotencia()
ParsearFactor()
VerificarSemanticaOperacion(List<TokenSintactico> tokensOp)
Match(params string[] esperados)
ReportarError(string mensaje)
ReportarErrorSemantico(string mensaje, int linea)

AnalizadorSemantico.Jerarquia

(24 métodos)

AnalizadorSemanticoJerarquia(...) [Constructor]
RegistrarVariable(string nombre, string tipoDato, string? valor = null)
TieneVariable(string nombre)
ObtenerTipo(string nombre) y ObtenerValor(string nombre)
ObtenerTablaTipos() y ObtenerTablaValores()
AnalizarExpresion(string expresionTexto, int linea = 1)
AnalizarTokensCompilador(List<TokenSintactico> tokensSintacticos)
Tokenizar(string entrada, int linea)
VerificarBalanceParentesis()
ParsearAsignacion()
ParsearLogico()
ParsearRelacional()
ParsearSumaResta()
ParsearMultiplicacionDivision()
ParsearPotencia()
ParsearPrimario()

Form1

(21 métodos)

Form1() [Constructor]
Form1_Load(object sender, EventArgs e)
ObtenerPalabrasDeLinea(string linea)
VerificarToken(int i, List<string> Tokens)
CargarEstructuraYDatosDesdeSQL()
richProgramaFuente_TextChanged(object sender, EventArgs e)
btnAnalizar_Click(object sender, EventArgs e)
AgregarErrores(string error, int linea)
ConfigurarDataGridView()
btnGuardarPrograma_Click(object sender, EventArgs e)
btnCargarPrograma_Click(object sender, EventArgs e)
btnEditarPrograma_Click(object sender, EventArgs e)
btnGuardarArchivo_Click(object sender, EventArgs e)
picLineas_Paint(...) y picLinea2_Paint(...)
richArchivoDeTokens_TextChanged(object sender, EventArgs e)
CompilarArbolJerarquia(AnalizadorSintactico? sintactico = null)

ObtenerTipoVariable(string nombreVar) y ObtenerValorVariable(string nombreVar)
InferirTiposYValores(NodoJerarquia nodo)
FormatearNumeroConSigno(double valor, bool esEntero)
GenerarPasosJerarquia(NodoJerarquia nodo)
ReportarError(string mensaje, int linea = 1)
CargarArbolEnTreeView(TreeView tv, NodoJerarquia? raiz, string tituloOperacion)
ExtraerOperacionesDeCodigoFuente(string codigoFuente)
CargarArbolSincronizado(TreeView tv, List<ResultadoSemanticoOperacion> operaciones)

ActualizarTablaDeSimbolos()
InicializarArbolEnEspera()
MostrarPasosOperacion(ResultadoSemanticoOperacion op)
cboOperacionesCodigo_SelectedIndexChanged(object sender, EventArgs e)
tvArbolJerarquia_AfterSelect(object sender, TreeViewEventArgs e)

Program

(1 métodos)

Main()

Arquitectura General del Pipeline del Compilador

El sistema implementa una arquitectura por etapas secuenciales fuertemente acopladas a través de estructuras de datos intermedias especializadas. A continuación se sintetiza el ciclo de vida del procesamiento desde el código fuente sin procesar hasta la ejecución y visualización del árbol sintáctico y semántico:

1. Fase Léxica (Autómata Finito Determinista con Persistencia en SQL Server)

El código fuente introducido en el RichTextBox o cargado desde disco es normalizado por `ObtenerPalabrasDeLinea` y procesado carácter a carácter por `VerificarToken`. Este método ejecuta transiciones de estado sobre `matrizCompleta` (descargada previamente de la base de datos NovaNyx en SQL Server). Reconoce palabras reservadas (RW01 a RW24), identificadores (IDV), constantes numéricas y de texto, operadores y caracteres especiales, registrando las nuevas variables en una lista simplemente enlazada y ordenada (`ClaseListaSimpleOrdenada`).

2. Fase Sintáctica (Parser Descendente Recursivo con Modo Pánico)

La secuencia de tokens generada es depurada de errores léxicos y recibida por `AnalizadorSintactico`. El método `ParsearPrograma` verifica los delimitadores obligatorios START y END, derivando en `ParsearInstrucciones`. Cada instrucción gramatical (IF/THEN/ELSE, WHILE, FOR, DO WHILE, EXECUTE, READ, PRINT, asignaciones) es procesada por métodos dedicados. En caso de fallos de sintaxis, `SincronizarHastaDelimitador` descarta tokens hasta el punto y coma más próximo para evitar cascadas de errores.

3. Fase Semántica y Árbol de Jerarquía (Parser de 7 Niveles e Inferencia de Tipos)

Cada expresión matemática o lógica detectada en asignaciones, condiciones o salidas es encapsulada y analizada por `AnalizadorSemanticoJerarquia`. Se construye un Árbol de Sintaxis Abstracta (AST) de acuerdo con los 7 niveles de precedencia matemática (Paréntesis, Potencia, Multiplicación/División, Suma/Resta, Relacional, Lógico, Asignación). El método `InferirTiposYValores` ejecuta un recorrido post-orden verificando compatibilidad de tipos (ej. prohibición de operar cadenas con enteros o división entre cero) y calculando valores constantes conocidos.

4. Fase de Presentación Visual y Persistencia en Disco

Los resultados de las fases anteriores convergen en la interfaz gráfica: `tvArbolJerarquia` despliega el árbol coloreado y con iconos mediante `ToTreeNode`; `rtbPasosJerarquia` detalla cada paso intermedio con `MostrarPasosOperacion`; `dgtTablaDeSimbolos` actualiza los tipos y valores calculados con `ActualizarTablaDeSimbolos`; y la clase `Archivo<Tipo>` asegura la persistencia en formato JSON de las fuentes y los tokens.

Módulo 1: Persistencia y Gestión de Flujos de Archivo (Archivo.cs)

Gestiona la persistencia de datos en disco mediante serialización JSON y control de flujos de texto (StreamReader/StreamWriter/FileStream), proporcionando operaciones seguras de apertura, lectura secuencial, escritura y eliminación de archivos para el código fuente y los tokens generados.

Archivo(string strArchivo)

PUBLIC[CONSTRUCTOR]ARCHIVO.CS

```
public Archivo(string strArchivo)
```

 PROPÓSITO Y ROL EN EL SISTEMA

Inicializa una nueva instancia de la clase genérica Archivo asignando el nombre y ruta del archivo sobre el cual se realizarán las operaciones de lectura o escritura.

 PARÁMETROS Y RETORNO

Parámetros: string strArchivo: Ruta o nombre del archivo en disco

Retorno: [Constructor]

 EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

El constructor recibe la ruta física o nombre del archivo `strArchivo` y lo almacena en la propiedad pública `NombreArchivo`. Esta propiedad es utilizada posteriormente por los métodos de control de flujo (`HacerModoEscritura`, `HacerModoLectura`, `EliminarArchivo`) para vincular el recurso del sistema operativo. No abre el flujo inmediatamente, permitiendo configurar el modo de apertura deseado de forma explícita.

Inicio: Archivo(strArchivo)

Recibir parámetro 'strArchivo'

Asignar this.NombreArchivo = strArchivo

Fin: Instancia Creada

DIAGRAMA DE FLUJO: ARCHIVO(STRING STRARCHIVO)

Crear()

PRIVATEVOIDARCHIVO.CS

```
private void Crear()
```

 PROPÓSITO Y ROL EN EL SISTEMA

Crea físicamente un nuevo archivo en el sistema de archivos con permisos de lectura y escritura concurrentes, inicializando el FileStream base.

 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

 EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Instancia un nuevo objeto `FileStream` pasando `NombreArchivo`, con el modo `FileMode.Create` (el cual sobrescribe el archivo si ya existe o lo crea si no existe) y el acceso `FileAccess.ReadWrite`. Este flujo queda asignado en el atributo privado `Flujo` para que los lectores o escritores de texto puedan operar sobre él.

Inicio: Crear()

Obtener this.NombreArchivo

Flujo = new FileStream(NombreArchivo, FileMode.Create, FileAccess.ReadWrite)

Fin: Flujo Inicializado

DIAGRAMA DE FLUJO: CREAM()

Página 5 de 58

~Archivo()

PROTECTED / INTERNAL

[DESTRUCTOR / FINALIZADOR]

ARCHIVO.CS

```
~Archivo()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Garantiza la liberación segura de los recursos del sistema operativo (manejadores de archivo no administrados) cuando el recolector de basura destruye la instancia.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: [Destructor / Finalizador]

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Invocado automáticamente por el Garbage Collector de .NET al recolectar la memoria del objeto. Llama directamente al método 'CerrarArchivo()', asegurando que el 'StreamReader', 'StreamWriter' y 'FileStream' sean cerrados y volcados al disco, previniendo bloqueos de archivos en Windows.



DIAGRAMA DE FLUJO: ~ARCHIVO()

HacerModoEscritura()

PUBLIC

VOID

ARCHIVO.CS

```
public void HacerModoEscritura()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Configura y abre el archivo en modo de adición y escritura, preparando el escritor de texto con volcado automático.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Comprueba mediante 'File.Exists(NombreArchivo)' si el archivo ya existe en disco.
2. Si existe, inicializa 'Flujo' con 'FileMode.Append' y 'FileAccess.Write' para preservar los datos existentes y escribir al final.
3. Si no existe, invoca al método privado 'Crear()' para generarlo.
4. Crea una instancia de 'StreamWriter(Flujo)' asignada a 'Escritor'.
5. Establece 'Escritor.AutoFlush = true' para asegurar que cada escritura se vacíe inmediatamente en el medio de almacenamiento sin quedar retenida en el búfer.

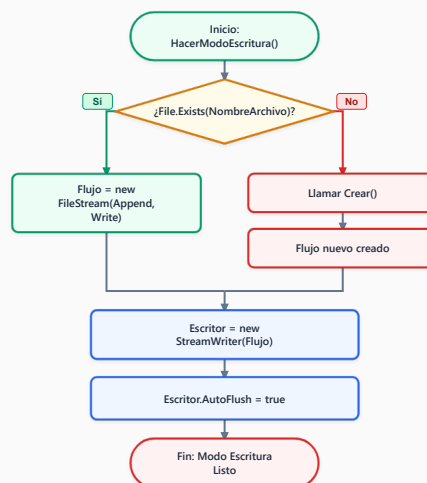


DIAGRAMA DE FLUJO: HACERMODOESCRITURA()

HacerModoLectura()

PUBLIC

VOID

ARCHIVO.CS

```
public void HacerModoLectura()
```

PROPÓSITO Y ROL EN EL SISTEMA

Prepara el archivo para operaciones de lectura secuencial inicializando el StreamReader asociado.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Verifica la existencia del archivo mediante 'File.Exists(NombreArchivo)'.
2. Si existe, abre el 'Flujo' en modo de apertura 'FileMode.Open' y acceso de solo lectura 'FileAccess.Read'.
3. Si no existe, invoca a 'Crear()' para crear un archivo vacío y evitar excepciones de tipo 'FileNotFoundException'.
4. Inicializa 'Lector = new StreamReader(Flujo)' permitiendo lecturas secuenciales línea por línea.

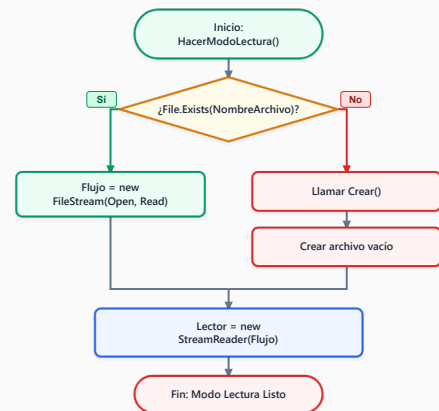


DIAGRAMA DE FLUJO: HACERMODELECTURA()

AgregarObjeto(Tipo Objeto)

PUBLIC

VOID

ARCHIVO.CS

```
public void AgregarObjeto(Tipo Objeto)
```

PROPÓSITO Y ROL EN EL SISTEMA

Serializa un objeto de tipo genérico en formato JSON y lo escribe como una nueva línea de texto en el archivo abierto.

PARÁMETROS Y RETORNO

Parámetros: Tipo Objeto: Entidad genérica a serializar y guardar

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Comprueba si 'Escritor == null'. En caso afirmativo, lanza una excepción 'InvalidOperationException' informando que el archivo no está en modo escritura.
2. Utiliza 'JsonSerializer.Serialize(Objeto)' para transformar la instancia en su representación textual JSON en una sola línea.
3. Invoca 'Escritor.WriteLine(json)' para escribir la línea en el archivo de texto.

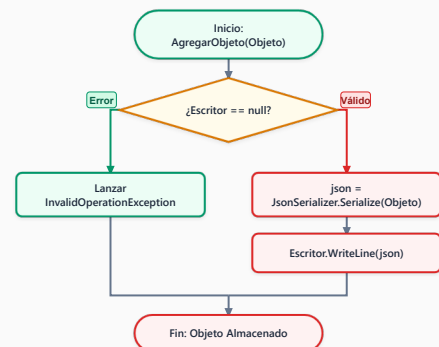


DIAGRAMA DE FLUJO: AGREGAROBJETO(TIPO OBJETO)

LeerObjeto()

PUBLIC

TIPO (OBJETO DESERIALIZADO O DEFAULT)

ARCHIVO.CS

```
public Tipo LeerObjeto()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Lee una línea del archivo en formato JSON y la deserializa nuevamente a la instancia original del tipo genérico Tipo.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: Tipo (Objeto deserializado o default)

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Valida si `Lector == null`. Si es nulo, arroja una excepción `InvalidOperationException`.
2. Comprueba la propiedad `FinArchivo`. Si se alcanzó el final, retorna el valor por defecto del tipo `default(Tipo)`.
3. Lee la siguiente línea con `Lector.ReadLine()`.
4. Si la cadena leída es nula o solo contiene espacios en blanco, retorna `default(Tipo)`.
5. Ejecuta `JsonSerializer.Deserialize<Tipo>(json)` y retorna la instancia reconstruida.

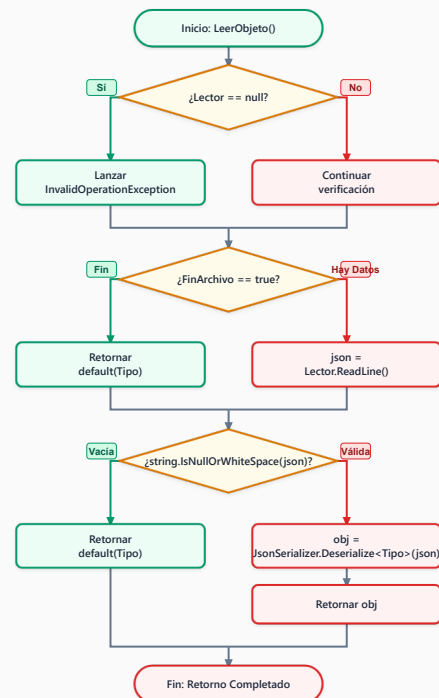


DIAGRAMA DE FLUJO: LEEROBJETO()

CerrarArchivo()

PUBLIC

VOID

ARCHIVO.CS

```
public void CerrarArchivo()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Cierra de manera ordenada y segura los lectores, escritores y flujos subyacentes liberando los descriptores del archivo.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Utiliza el operador de navegación segura `?.` para invocar `Close()` en cada componente:

1. `Lector?.Close()`: Cierra el lector y libera el búfer de lectura.
2. `Escritor?.Close()`: Vacía y cierra el escritor de texto.
3. `Flujo?.Close()`: Libera el manejador del `FileStream` del sistema operativo Windows.

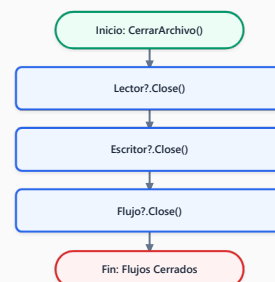


DIAGRAMA DE FLUJO: CERRARARCHIVO()

EliminarArchivo()

PUBLIC

VOID

ARCHIVO.CS

```
public void EliminarArchivo()
```

PROPÓSITO Y ROL EN EL SISTEMA

Cierra los flujos activos y elimina de forma definitiva el archivo físico del disco duro.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Llama obligatoriamente a `CerrarArchivo()` para liberar los bloqueos de lectura/escritura en Windows.
2. Comprueba si el archivo existe con `File.Exists(NombreArchivo)`.
3. Si existe, llama a `File.Delete(NombreArchivo)`.
4. Si no existe en el sistema, lanza una excepción de tipo `Exception` informando que el archivo a eliminar no existe.

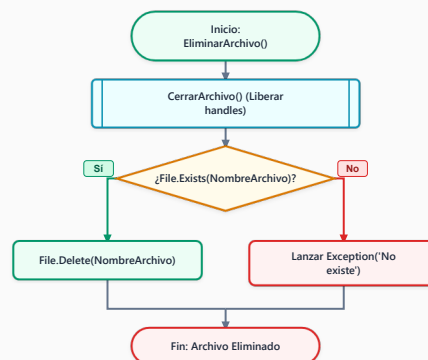


DIAGRAMA DE FLUJO: ELIMINARARCHIVO()

FinArchivo [Propiedad get]

PUBLIC

BOOL

ARCHIVO.CS

```
public bool FinArchivo { get; }
```

PROPÓSITO Y ROL EN EL SISTEMA

Indica si el cursor de lectura ha alcanzado el final del archivo o si no hay un flujo activo.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: bool

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si `Lector != null`, consulta la propiedad `Lector.EndOfStream` y la retorna.
2. Si `Flujo != null`, compara si `Flujo.Position >= Flujo.Length`.
3. Si tanto el lector como el flujo son nulos, retorna `true` por defecto indicando que no hay más datos disponibles.

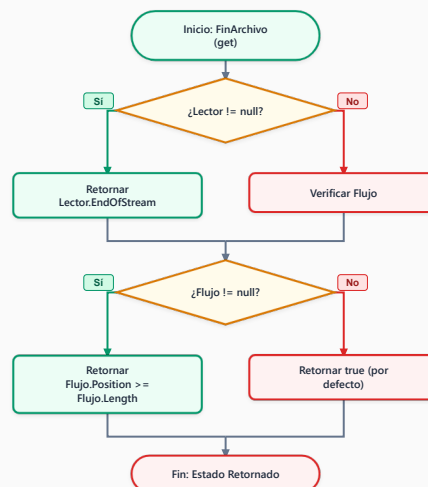


DIAGRAMA DE FLUJO: FINARCHIVO [PROPIEDAD GET]

Módulo 2: Estructura de Nodo Doblemente Enlazado (ClaseNodo.cs)

Proporciona la estructura atómica fundamental para colecciones enlazadas en memoria. Contiene un objeto de tipo genérico junto con punteros bidireccionales de enlace ('Anterior' y 'Siguiente'), asegurando la desvinculación segura de referencias durante la recolección de basura.

~ClaseNodo()

PROTECTED / INTERNAL

[DESTRUCTOR / FINALIZADOR]

CLASENODO.CS

~ClaseNodo()

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Libera la referencia del objeto almacenado dentro del nodo al ser recolectado por el recolector de basura.

📄 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: [Destructor / Finalizador]

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Al ejecutarse la finalización del objeto por el Garbage Collector, este destructor restablece la propiedad 'ObjetoConDatos = default(Tipo)', desvinculando la referencia en memoria y facilitando la pronta recolección del objeto subyacente.

Inicio: ~ClaseNodo()

Asignar ObjetoConDatos = default(Tipo)

Fin: Referencia Liberada

DIAGRAMA DE FLUJO: ~CLASENODO()

ObjetoConDatos, Anterior, Siguiente [Propiedades]

PUBLIC

TIPO / CLASENODO<TIPO>

CLASENODO.CS

public Tipo ObjetoConDatos { get; set; }
public ClaseNodo<Tipo> Anterior { get; set; }
public ClaseNodo<Tipo> Siguiente { get; set; }

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Provee los métodos de acceso get y set para manipular el dato almacenado y los punteros bidireccionales en la estructura de nodo enlazado.

📄 PARÁMETROS Y RETORNO

Parámetros: value: Valor a asignar
Retorno: Tipo / ClaseNodo<Tipo>

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Encapsula los campos privados '_objetoConDatos', '_anterior' y '_siguiente'. Permite tanto a listas simples como doblemente enlazadas acceder al dato contenido y enlazar los nodos previo y posterior en la cadena de memoria.

Inicio: Acceso a Propiedades

¿Operación Get o Set?

Get: Retornar campo privado correspondiente

Set: Asignar campo = value

Fin: Operación Completada

DIAGRAMA DE FLUJO: OBJETOCONDATOS, ANTERIOR, SIGUIENTE [PROPIEDADES]

Página 10 de 58

Módulo 3: Entidad de Tabla de Símbolos (Identificador.cs)

Representa una variable o identificador detectado dentro del código fuente durante el análisis léxico. Almacena su índice correlativo único, su nombre, tipo de dato deducido y valor actual calculado, e implementa IEquatable e IComparable para ordenación eficiente.

Equals(Identificador other)

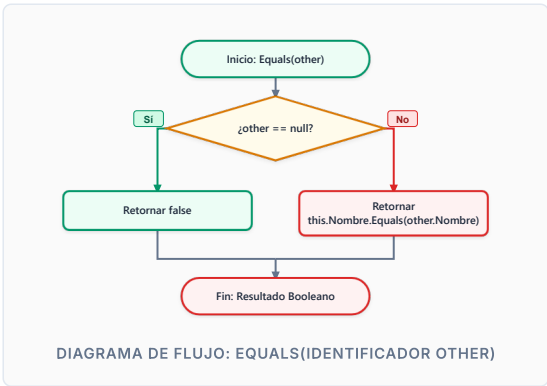
PUBLIC BOOL IDENTIFICADOR.CS

```
public bool Equals(Identificador other)
```

PROPÓSITO Y ROL EN EL SISTEMA
Determina si dos identificadores son lógicamente iguales comparando sus nombres léxicos.

PARÁMETROS Y RETORNO
Parámetros: Identificador other: Objeto identificador con el que comparar
Retorno: bool

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO
Implementa la interfaz estándar 'IEquatable<Identificador>'. Invoca 'this.Nombre.Equals(other.Nombre)'. Permite a la lista ordenada y colecciones detectar rápidamente colisiones o duplicados de variables en la tabla de símbolos basándose exclusivamente en el lexema del identificador.



CompareTo(Identificador other)

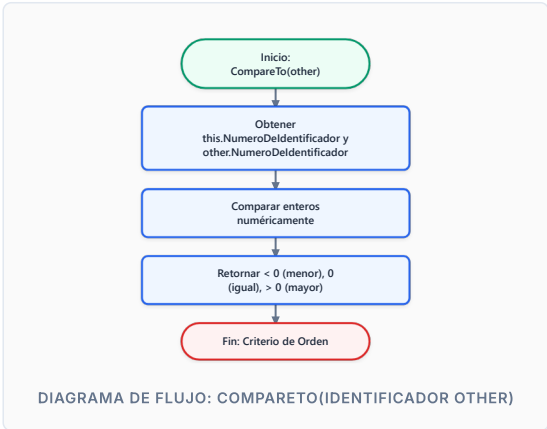
PUBLIC INT (-1, 0, 1) IDENTIFICADOR.CS

```
public int CompareTo(Identificador other)
```

PROPÓSITO Y ROL EN EL SISTEMA
Compara el identificador actual con otro según su número entero único asignado (NumeroDelIdentificador).

PARÁMETROS Y RETORNO
Parámetros: Identificador other: Objeto con el cual comparar el orden numérico
Retorno: int (-1, 0, 1)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO
Implementa 'IComparable<Identificador>'. Llama a 'this.NumeroDelIdentificador.CompareTo(other.NumeroDelIdentificador)'. Proporciona el criterio de ordenamiento ascendente estricto utilizado por la lista simplemente ordenada al insertar nuevos identificadores en la tabla de símbolos.



Módulo 4: Árbol de Jerarquía y Precedencia Semántica (NodoJerarquia.cs)

Modela los nodos del árbol de sintaxis abstracta y precedencia jerárquica de expresiones (niveles 1 a 7). Integra algoritmos de conversión recursiva a controles visuales TreeNode con iconos y colores temáticos según el operador y su estado semántico.

NodoJerarquia() y Sobrecarga con Parámetros

PUBLIC

[CONSTRUCTORES]

NODOJERARQUIA.CS

```
public NodoJerarquia()  
public NodoJerarquia(string lexema, string tipoNodo, int nivel, string descripcion, string tipoDato = "desconocido")
```

🎯 PROPÓSITO Y ROL EN EL SISTEMA

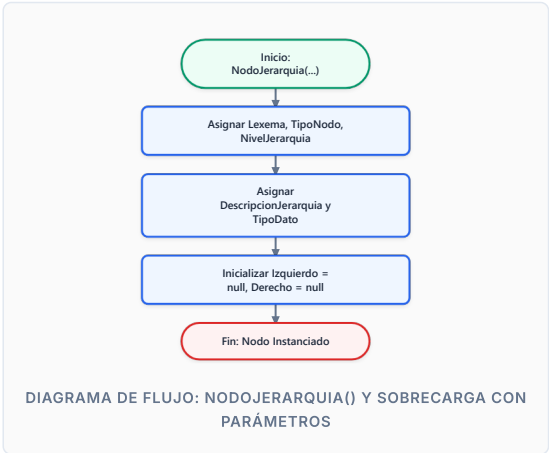
Inicializa un nodo del árbol de jerarquía semántica con toda su información sintáctica, jerárquica y de tipos de datos.

📄 PARÁMETROS Y RETORNO

Parámetros: lexema: Símbolo o valor del nodo; tipoNodo: Categoría (Operador, Constante, etc.); nivel: Prioridad (1 a 7); descripcion: Texto explicativo; tipoDato: Tipo semántico
Retorno: [Constructores]

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

El constructor por defecto inicializa campos vacíos. El constructor parametrizado inicializa las propiedades 'Lexema', 'TipoNodo' ('Asignacion', 'Operador', 'Agrupacion', 'Identificador', 'Constante'), 'NivelJerarquia' (prioridad 1 a 7), 'DescripcionJerarquia' y 'TipoDato'. Los subárboles 'Izquierdo' y 'Derecho' quedan inicialmente en null para su posterior enlace durante el análisis.



```
public TreeNode ToTreeNode()
```

🎯 PROPÓSITO Y ROL EN EL SISTEMA

Convierte recursivamente el subárbol semántico en un componente visual TreeNode con etiquetas descriptivas, iconos temáticos y colores según la prioridad.

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: TreeNode (Nodo visual para TreeView)

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Evalúa si el nodo tiene error semántico ('TieneError'). Si es así, antepone '✖' [ERROR SEMÁNTICO] y colorea el texto de rojo.
2. Si es válido, evalúa 'TipoNode':
 - Agrupación: '🟡' [Nivel 1: Paréntesis] en DarkMagenta.
 - Asignación: '🎯' [Nivel 7: Asignación] en Navy.
 - Operador: Iconos según nivel (⚡ Potencia, ✖/÷ Mult/Div, ⚡/= Suma/Resta, 🛡 Relacional, 📄 Lógico) coloreados acordeamente.
 - Constante o Identificador: Prefijos '📄' o '📄'.
3. Concatena el resultado evaluado si existe ('ValorCalculado').
4. Llama recursivamente a 'ToTreeNode()' sobre 'Izquierdo', 'Derecho' y la lista de 'Hijos', agregándolos a 'treeNode.Nodes'.
5. Retorna el nodo gráfico raíz configurado.

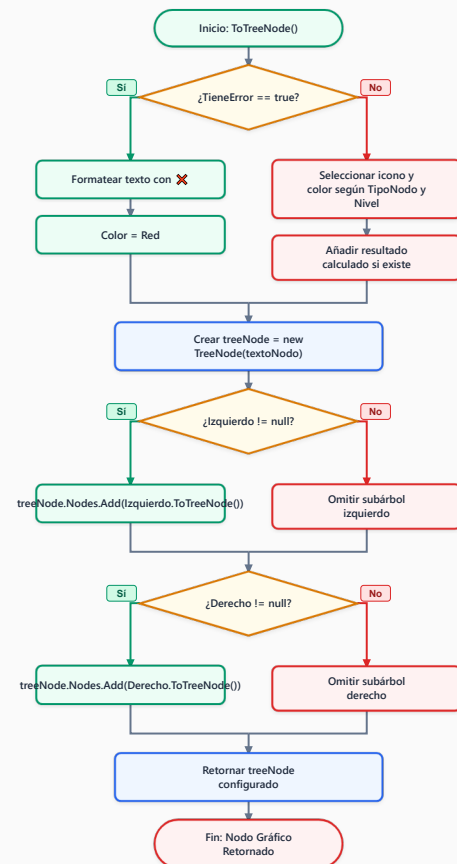


DIAGRAMA DE FLUJO: TOTREENODE()

Módulo 5: Lista Enlazada Genérica Ordenada (ClaseListaSimpleOrdenada.cs)

Implementa una lista lineal enlazada genérica que mantiene automáticamente todos sus elementos ordenados ascendentemente mediante la interfaz IComparable<Tipo>. Impide la inserción de duplicados y optimiza las operaciones de búsqueda y eliminación.

ClaseListaSimpleOrdenada() [Constructor]

PUBLIC [CONSTRUCTOR] CLASELISTASIMPLEORDENADA.CS

```
public ClaseListaSimpleOrdenada()
```

PROPÓSITO Y ROL EN EL SISTEMA

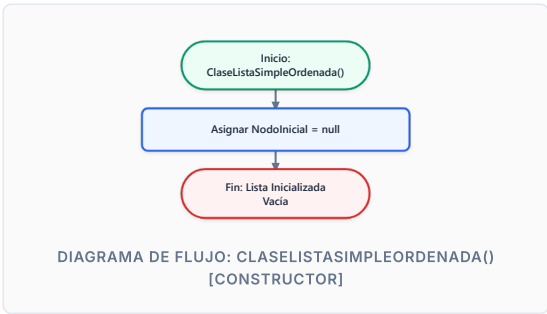
Crea una nueva lista lineal simplemente enlazada estableciendo la referencia de inicio en null.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: [Constructor]

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Inicializa el atributo 'NodoInicial = null'. Deja la lista en estado vacío (con la propiedad 'Vacía' evaluando a true), lista para recibir inserciones ordenadas mediante algoritmos basados en IComparable.



GetEnumerator()

PUBLIC [ENUMERATOR<TIPO>] CLASELISTASIMPLEORDENADA.CS

```
public IEnumerator<Tipo> GetEnumerator()
```

PROPÓSITO Y ROL EN EL SISTEMA

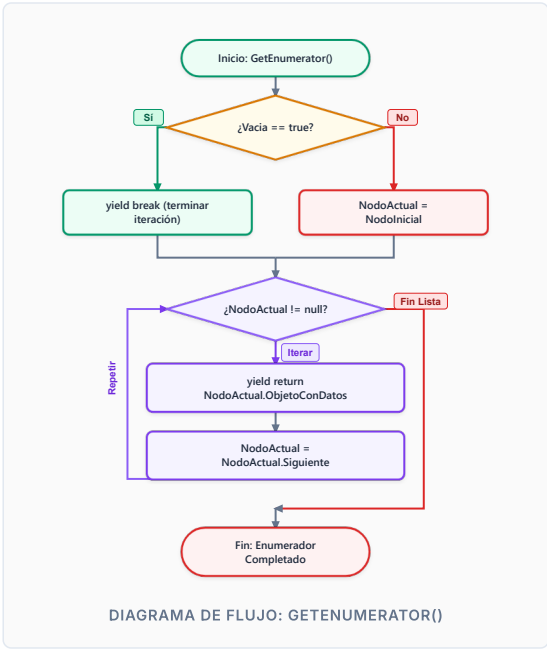
Permite iterar secuencialmente a través de los datos almacenados en la lista utilizando bucles foreach.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: IEnumerator<Tipo>

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Evalúa si la lista está vacía ('Vacía == true'). Si es así, termina la enumeración inmediatamente mediante 'yield break'.
2. En caso contrario, establece un puntero local 'NodoActual = NodoInicial'.
3. En un bucle 'do-while', retorna el objeto con 'yield return (NodoActual.ObjetoConDatos)' y avanza con 'NodoActual = NodoActual.Siguiente' hasta alcanzar el final de la lista ('null').



Insertar(Tipo Objeto)

PUBLIC

VOID

CLASELISTASIMPLEORDENADA.CS

```
public void Insertar(Tipo Objeto)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Inserta un nuevo elemento en orden ascendente según CompareTo y previene duplicados en la lista.

🔗 PARÁMETROS Y RETORNO

Parámetros: Tipo Objeto: Elemento que se insertará de forma ordenada en la lista
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si la lista está vacía ('Vacía'), crea un nuevo 'ClaseNodo<Tipo>', le asigna el dato y lo define como 'NodoInicial'.
2. Si contiene nodos, inicia un recorrido con 'NodoActual = NodoInicial' y 'NodoPrevio'.
3. Si 'Objeto.Equals(NodoActual.ObjetoConDatos)', lanza excepción: 'No se aceptan duplicados en la lista'.
4. Si 'Objeto.CompareTo(NodoActual.ObjetoConDatos) < 0', encontró la posición de inserción:
 - Si es el primer nodo ('NodoActual == NodoInicial'), inserta a la cabeza actualizando 'NodoInicial'.
 - Si es intermedio, enlaza 'NodoPrevio.Siguiente = NodoNuevo' y 'NodoNuevo.Siguiente = NodoActual'.
5. Si no es menor, avanza los punteros hasta llegar al final y lo enlaza al último nodo.

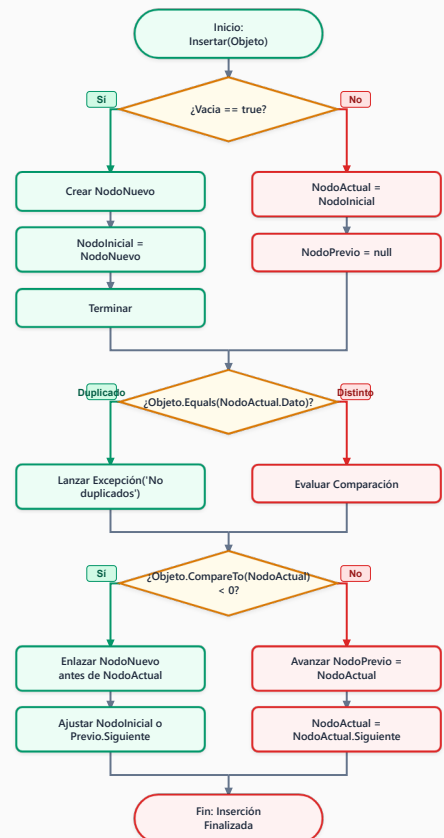


DIAGRAMA DE FLUJO: INSERTAR(TIPO OBJETO)

Eliminar(Tipo Objeto)

PUBLIC

TIPO (OBJETO ELIMINADO DE LA LISTA)

CLASELISTASIMPLEORDENADA.CS

```
public Tipo Eliminar(Tipo Objeto)
```

PROPÓSITO Y ROL EN EL SISTEMA

Busca un elemento específico en la lista ordenada, lo desenlaza de la estructura y retorna el valor eliminado.

PARÁMETROS Y RETORNO

Parámetros: Tipo Objeto: Elemento que se desea buscar y remover de la lista
Retorno: Tipo (Objeto eliminado de la lista)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Valida si la lista está vacía ('Vacía'). Si es así, lanza excepción: 'La lista esta vacía'.
2. Si el objeto coincide con 'NodoInicial.ObjetoConDatos', guarda el dato en 'Auxiliar', mueve la cabeza a 'NodoActual.Siguiente' y retorna el dato.
3. Si coincide con un nodo intermedio, enlaza 'NodoPrevio.Siguiente = NodoActual.Siguiente', desvinculando el nodo actual.
4. Debido a que la lista está ordenada, si 'Objeto.CompareTo(NodoActual.ObjetoConDatos) < 0', significa que el elemento ya no puede existir más adelante, por lo que interrumpe la búsqueda lanzando la excepción 'No existe dentro de la lista'.
5. Si alcanza el fin sin hallarlo, lanza la excepción de no existencia.

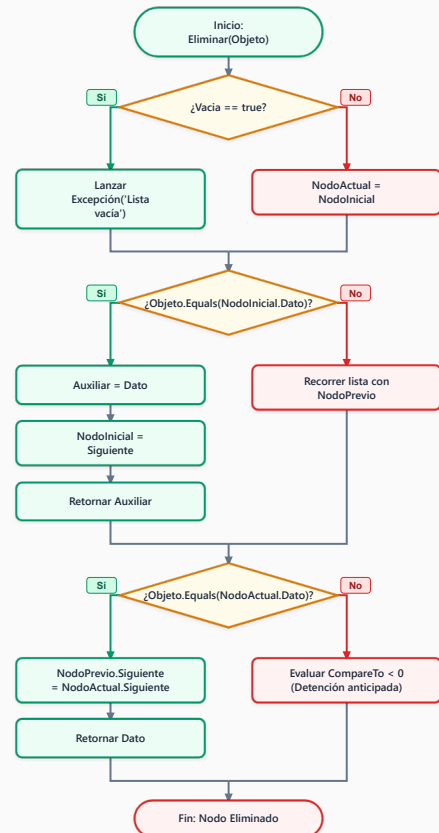


DIAGRAMA DE FLUJO: ELIMINAR(TIPO OBJETO)

Buscar(Tipo Objeto)

PUBLIC

BOOL (TRUE SI EXISTE, FALSE EN CASO CONTRARIO)

CLASELISTASIMPLEORDENADA.CS

```
public bool Buscar(Tipo Objeto)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Determina con eficiencia de lista ordenada si un objeto se encuentra presente en la lista.

🔧 PARÁMETROS Y RETORNO

Parámetros: Tipo Objeto: Elemento buscado

Retorno: bool (true si existe, false en caso contrario)

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si 'Vacía', lanza excepción: 'La lista esta vacia'.
2. Recorre secuencialmente comparando con 'Equals'.
3. Si encuentra coincidencia con el nodo actual, retorna 'true'.
4. Si 'Objeto.CompareTo(NodoActual.ObjetoConDatos) < 0', detiene la búsqueda y retorna 'false' de inmediato aprovechando el orden ascendente, evitando iterar innecesariamente sobre el resto de la lista.
5. Si finaliza el recorrido completo, retorna 'false'.

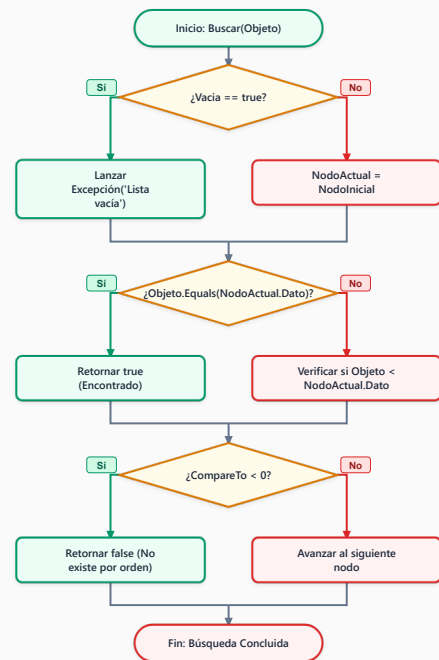


DIAGRAMA DE FLUJO: BUSCAR(TIPO OBJETO)

```
public void Vaciarse()
~ClaseListaSimpleOrdenada()
```

PROPÓSITO Y ROL EN EL SISTEMA

Desvincula secuencialmente cada uno de los nodos enlazados, dejando la lista vacía y facilitando la liberación de memoria.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si 'Vacía', lanza excepción 'La lista esta vacía' (cuando es llamado manualmente desde Vaciarse()).
2. Recorre la lista asignando 'NodoPrevio = NodoActual', avanzando 'NodoActual = NodoActual.Siguiente' y rompiendo los enlaces con 'NodoPrevio = null'.
3. Al finalizar el ciclo, establece 'NodoInicial = null'.
4. El destructor finalizador '~ClaseListaSimpleOrdenada()' invoca 'Vaciarse()' para garantizar que la memoria se limpie al destruir la lista.

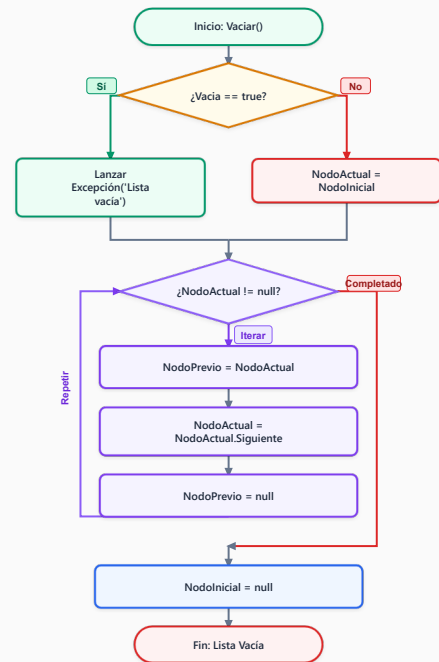


DIAGRAMA DE FLUJO: VACIARSE() Y ~CLASELISTASIMPLEORDENADA()

Contar [Propiedad get]

PUBLIC

INT (CANTIDAD DE ELEMENTOS)

CLASELISTASIMPLEORDENADA.CS

```
public int Contar { get; }
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Calcula de forma dinámica el número total de nodos que componen la lista enlazada.

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: int (Cantidad de elementos)

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si '!'Vacía', inicializa un acumulador 'Contador = 0' y sitúa 'NodoActual = NodoInicial'.
2. Itera secuencialmente mediante un bucle incrementando 'Contador++' y avanzando a 'NodoActual.Siguiente' hasta alcanzar el puntero nulo.
3. Retorna el valor acumulado en 'Contador'.
4. Si la lista está vacía, retorna 0 directamente.

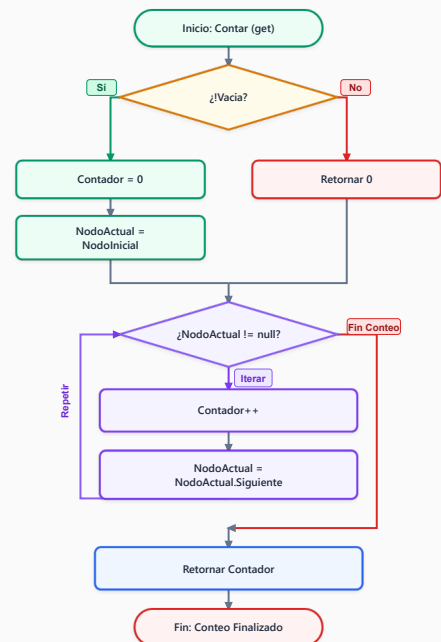


DIAGRAMA DE FLUJO: CONTAR [PROPIEDAD GET]

Módulo 6: Analizador Sintáctico Descendente Recursivo (AnalizadorSintactico.cs)

Implementa el parser descendente recursivo de la gramática formal del lenguaje NovaNyx 1.4.2. Valida la estructura general del programa (START ... END), bloques condicionales (IF/THEN/ELSE), bucles (WHILE, FOR, DO, EXECUTE), selección múltiple (PERHAPS/CASE), instrucciones de I/O (READ, PRINT) y sincronización de errores en modo pánico.

AnalizadorSintactico(...) [Constructor]

PUBLIC

[CONSTRUCTOR]

ANALIZADORSINTACTICO.CS

```
public AnalizadorSintactico(List<TokenSintactico> tokens, DataGridView dgtErrores, AnalizadorSemanticoJerarquia? semantico = null)
```

PROPÓSITO Y ROL EN EL SISTEMA

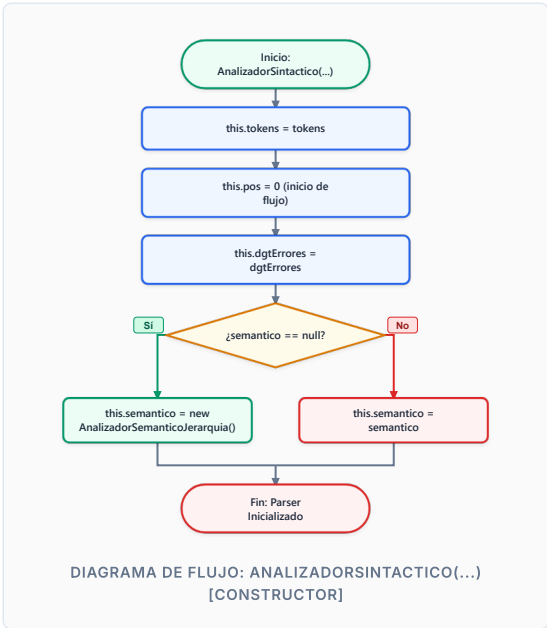
Inicializa el parser sintáctico descendente recursivo vinculando la lista de tokens, la cuadrícula de errores y el motor semántico.

PARÁMETROS Y RETORNO

Parámetros: tokens: Lista de tokens sintácticos; dgtErrores: Control visual para reportar anomalías; semantico: Instancia del evaluador semántico y jerárquico
Retorno: [Constructor]

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Guarda la referencia a la lista de tokens a analizar, coloca el índice de lectura 'pos = 0', asocia el 'DataGridView' para inserción directa de filas de error y configura la instancia de 'AnalizadorSemanticoJerarquia' (si se recibe null, crea una nueva automáticamente). Inicializa además la lista pública 'OperacionesDetectadas' para almacenar cada expresión encontrada.



EsDelimitador(TokenSintactico tok)

PRIVATE

BOOL (TRUE SI ES DELIMITADOR, FALSE EN CASO CONTRARIO)

ANALIZADORSINTACTICO.CS

```
private bool EsDelimitador(TokenSintactico tok)
```

PROPÓSITO Y ROL EN EL SISTEMA

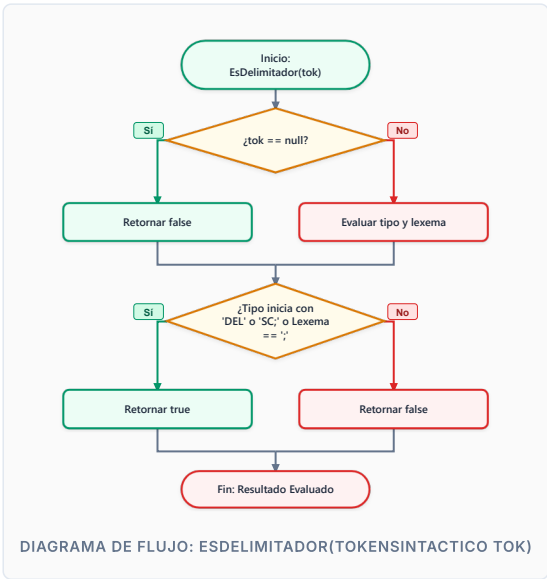
Determina de forma unificada si un token representa un delimitador de fin de instrucción o punto y coma.

PARÁMETROS Y RETORNO

Parámetros: TokenSintactico tok: Token a evaluar
Retorno: bool (true si es delimitador, false en caso contrario)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si 'tok == null', retorna 'false'.
2. Evalúa si 'tok.Tipo' comienza con 'DEL' (clasificación léxica de delimitadores).
3. Evalúa si 'tok.Tipo' comienza con 'SC;' (código de carácter especial punto y coma).
4. Evalúa si 'tok.Lexema == ';'.
5. Si se cumple cualquiera de estas condiciones, retorna 'true'; de lo contrario, 'false'.



MatchDelimitador()

PRIVATE

BOOL (TRUE SI CONSUMIÓ DELIMITADOR, FALSE SI NO)

ANALIZADORSINTACTICO.CS

```
private bool MatchDelimitador()
```

PROPÓSITO Y ROL EN EL SISTEMA

Consume de forma tolerante uno o más delimitadores consecutivos en la posición actual del cursor de tokens.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: bool (true si consumió delimitador, false si no)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Comprueba si el índice 'pos >= tokens.Count'. Si es así, retorna 'false'.
2. Si el token actual es un delimitador según 'EsDelimitador(tokens[pos])', entra en un bucle 'while' consumiendo todos los delimitadores consecutivos ('pos++'), evitando fallos por puntos y coma repetidos (ej. ';;').
3. Retorna 'true' si al menos un delimitador fue consumido, o 'false' en caso contrario.

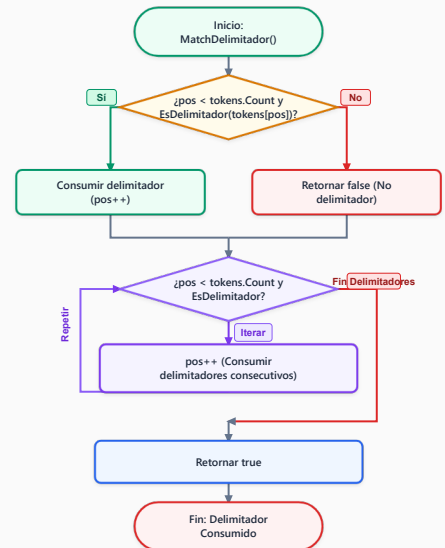


DIAGRAMA DE FLUJO: MATCHDELIMITADOR()

ExigirDelimitador(string contexto = "")

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ExigirDelimitador(string contexto = "")
```

PROPÓSITO Y ROL EN EL SISTEMA

Verifica obligatoriamente la presencia de un delimitador al final de una instrucción y emite un error si falta.

PARÁMETROS Y RETORNO

Parámetros: string contexto: Nombre o descripción de la instrucción para personalizar el mensaje de error

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Llama a 'MatchDelimitador()'.
2. Si retorna 'false' (no se encontró delimitador), construye un mensaje explicativo contextualizado (ej. 'Se esperaba delimitador ';' (DEL) al final de la asignación').
3. Invoca a 'ReportarError(msg)' para alertar al usuario en la cuadrícula de errores sin interrumpir la ejecución del resto del programa.

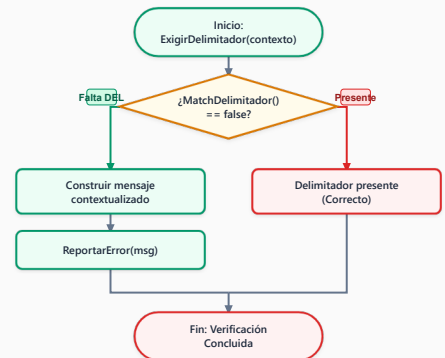


DIAGRAMA DE FLUJO: EXIGIRDELIMITADOR(STRING CONTEXTO = "")

SincronizarHastaDelimitador()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void SincronizarHastaDelimitador()
```

🎯 PROPÓSITO Y ROL EN EL SISTEMA

Implementa el modo pánico de recuperación de errores sintácticos, descartando tokens hasta encontrar un delimitador seguro.

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Cuando ocurre un error grave en una instrucción, este método itera avanzando 'pos++' mientras no se alcance el final de los tokens, no se encuentre un delimitador y no sea el fin de un bloque de código ('!EsFinDeBloque()').
2. Si encuentra el delimitador, lo consume ('pos++').
3. Esto sincroniza al parser con la siguiente sentencia válida, previniendo la generación de cascadas interminables de errores espurios.

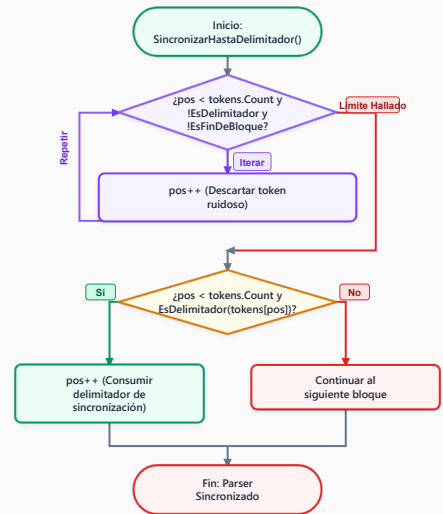


DIAGRAMA DE FLUJO: SINCRONIZARHASTADELIMITADOR()

ParsearPrograma()

PUBLIC

VOID

ANALIZADORSINTACTICO.CS

```
public void ParsearPrograma()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Punto de entrada general del análisis sintáctico que valida la envoltura global del programa (START ... END).

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Ignora delimitadores en blanco iniciales.
2. Si `pos >= tokens.Count`, reporta error de programa vacío.
3. Valida la presencia obligatoria de la palabra reservada de inicio 'START' (RW01). Si no está, emite error.
4. Consume delimitador opcional tras START.
5. Invoca a `ParsearInstrucciones()` para analizar todo el cuerpo del código fuente.
6. Exige la palabra reservada de fin 'END' (RW02).
7. Consume delimitador opcional tras END.
8. Verifica que no queden tokens flotantes fuera de la estructura principal después de END, reportando error si existen.

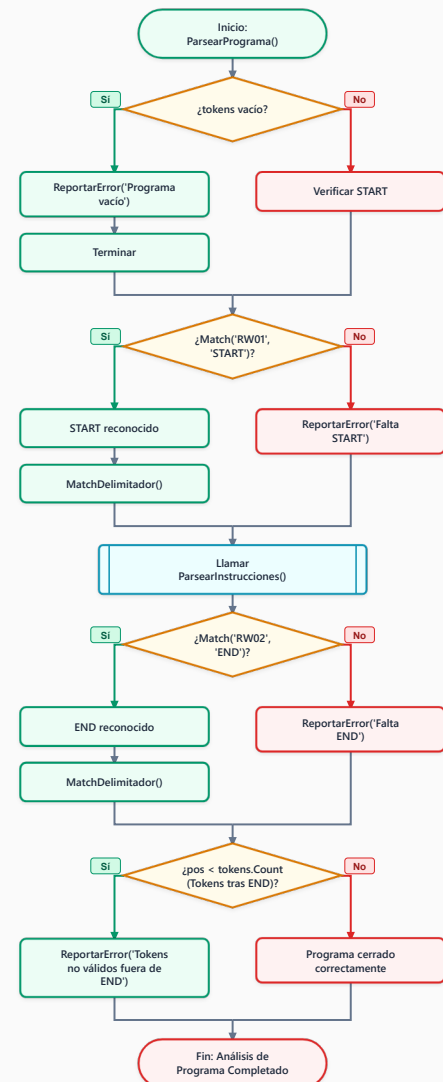


DIAGRAMA DE FLUJO: PARSEARPROGRAMA()

ParsearInstrucciones(params string[] stopTokensAdicionales)

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearInstrucciones(params string[] stopTokensAdicionales)
```

PROPÓSITO Y ROL EN EL SISTEMA

Procesa secuencialmente un bloque de sentencias hasta encontrar un token de cierre de estructura.

PARÁMETROS Y RETORNO

Parámetros: params string[] stopTokensAdicionales: Tokens especiales que marcan el final de un bloque interno
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Ejecuta un ciclo `while` mientras el índice no supere el total de tokens y la condición `!EsFinDeBloque(stopTokensAdicionales)` sea verdadera. En cada ciclo delega la interpretación de la sentencia a `ParsearInstruccion()`. Al encontrarse palabras como `ENDIF`, `ELSE`, `ENDWHILE`, `ENDFOR` o tokens de parada opcionales, el método finaliza ordenadamente cediendo el control a la estructura llamadora.

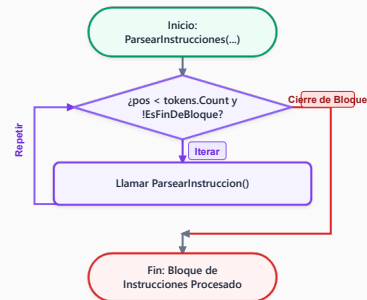


DIAGRAMA DE FLUJO: PARSEARINSTRUCCIONES(PARAMS STRING[] STOPTOKENSADICIONALES)

EsFinDeBloque(string[]? stopTokensAdicionales = null)

PRIVATE

BOOL (TRUE SI EL TOKEN ACTUAL CIERRA UN BLOQUE)

ANALIZADORSINTACTICO.CS

```
private bool EsFinDeBloque(string[]? stopTokensAdicionales = null)
```

PROPÓSITO Y ROL EN EL SISTEMA

Determina si el token que apunta el cursor actual corresponde al delimitador de cierre de un bloque gramatical.

PARÁMETROS Y RETORNO

Parámetros: stopTokensAdicionales: Lista opcional de tokens de parada personalizados
Retorno: bool (true si el token actual cierra un bloque)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- Si `pos >= tokens.Count`, retorna `true`.
- Si se suministraron `stopTokensAdicionales`, itera sobre ellos y retorna `true` si el tipo o lexema coincide (por ejemplo, el `WHILE` de un `DO ... WHILE` o `UNTIL` de `EXECUTE`).
- Comprueba las palabras reservadas de cierre estándar en la gramática NovaNyx 1.4.2: `END` (RW02), `ELSE` (RW07), `FALSE` (RW08), `ENDIF` (RW09), `CASE` (RW11), `NONE` (RW12), `ENDCASE` (RW13), `ENDFOR` (RW19), `ENDDO` (RW23), `ENDWHILE` (RW24).
- Retorna `true` si coincide con alguna, o `false` en caso contrario.

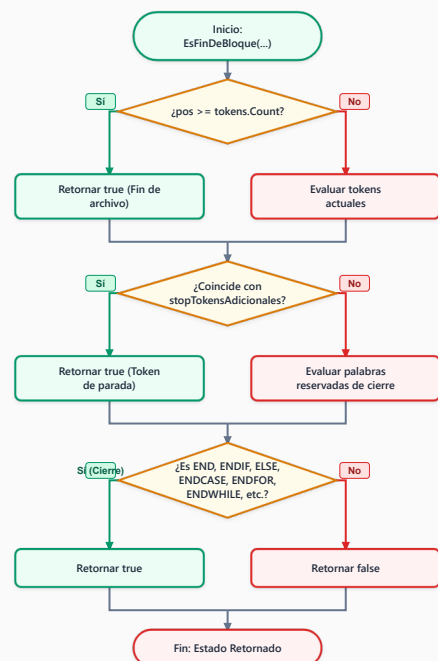


DIAGRAMA DE FLUJO: ESFINDEBLOQUE(STRING[]? STOPTOKENSADICIONALES = NULL)

ParsearInstruccion()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearInstruccion()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Discrimina el tipo de instrucción según el token actual y ejecuta la validación de la gramática correspondiente.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Evalúa el tipo y lexema del token actual:

- Asignación ('IDV'): Valida 'IDV = Expresion;', extrayendo la subsecuencia de tokens y enviándola a 'VerificarSemanticaOperacion'.
- Lectura ('READ' / RW03): Valida argumentos válidos (variables o constantes) separados por comas y finalizados con delimitador.
- Escritura ('PRINT' / RW04): Valida una o más expresiones separadas por comas, verificando su semántica.
- Condición 'IF' (RW05): Valida 'IF Condicion THEN Instrucciones [ELSE/FALSE Instrucciones] ENDIF;'.
- Selección Múltiple 'PERHAPS' (RW10): Valida expresión selector, casos 'CASE Condicion: Instrucciones', bloque por defecto 'NONE:' y cierre 'ENDCASE;'.
- Ciclo 'WHILE' (RW20): Valida 'WHILE Condicion [DO] Instrucciones ENDWHILE;'.
- Ciclo 'FOR' (RW14): Valida formas 'FOR [FROM] id = inicio UNTIL fin INTERVAL paso Instrucciones ENDFOR;'.
- Ciclos 'DO WHILE' (RW21) y 'EXECUTE' (RW22): Valida ejecuciones repetitivas y sus condiciones terminales.
- Si no coincide con ninguna estructura conocida, reporta error de instrucción no válida y ejecuta 'SincronizarHastaDelimitador()'.

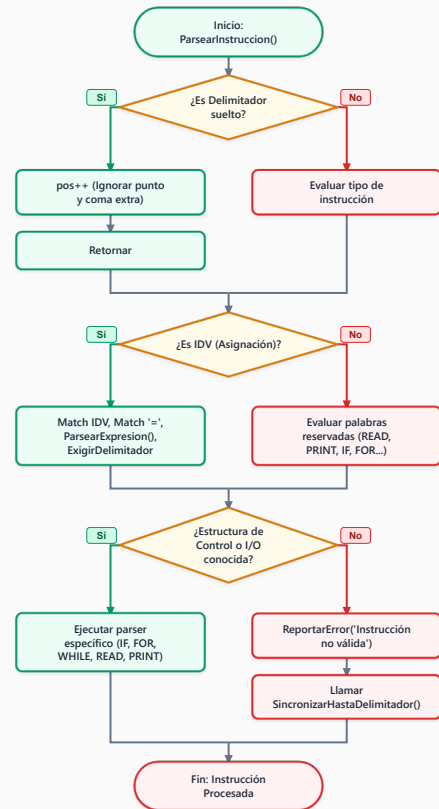


DIAGRAMA DE FLUJO: PARSEARINSTRUCCION()

ParsearCondicion()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearCondicion()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Valida la sintaxis de condiciones lógicas y relacionales compuestas utilizadas en IF, WHILE, UNTIL, etc.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Guarda la posición inicial 'inicioCond = pos' para capturar los tokens.
2. Comprueba si existe operador unario 'NOT' (LO2) y lo consume.
3. Soporta subcondiciones agrupadas entre paréntesis '(Condicion)'.
4. Si no tiene paréntesis, invoca 'ParsearExpresion()' para el operando izquierdo.
5. Verifica la presencia de un operador relacional ('>', '<', '==', '!=', '>=', '<=') y analiza el operando derecho con 'ParsearExpresion()'.
6. Comprueba operadores lógicos adicionales ('AND', 'OR') invocando recursivamente a 'ParsearCondicion()'.
7. Si no se halló ningún operador relacional ni lógico, emite error sintáctico.
8. Envía la subsecuencia de tokens a 'VerificarSemanticaOperacion' para construir su árbol jerárquico.

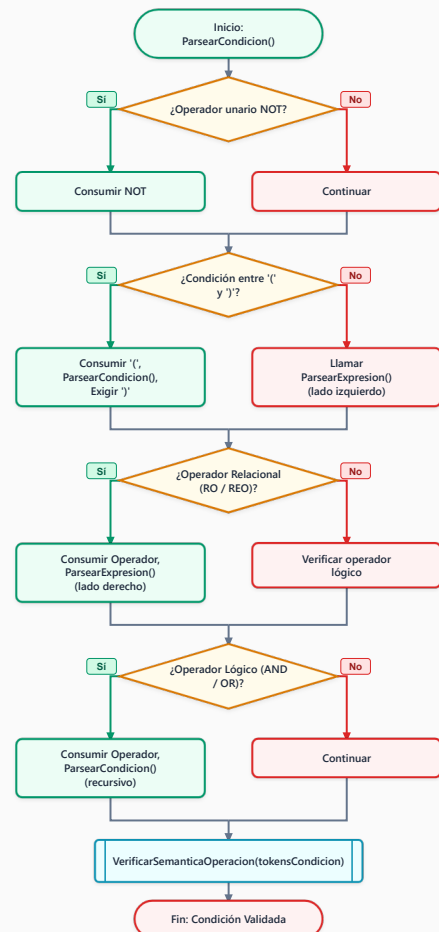


DIAGRAMA DE FLUJO: PARSEARCONDICION()

ParsearCondicionOExpresion()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearCondicionOExpresion()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Analiza argumentos mixtos que pueden presentarse como una expresión aritmética simple o una condición relacional (ej. cláusulas CASE o UNTIL).

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- Guarda el índice de inicio 'inicio = pos'.
- Llama a 'ParsearExpresion()'.
- Comprueba si a continuación existe un operador relacional ('RO' o 'REO'). De ser así, lo consume y procesa la segunda expresión con 'ParsearExpresion()'.
- Si se capturaron tokens, los envía a 'VerificarSemanticaOperacion' para registrar la operación en la lista de análisis semántico.

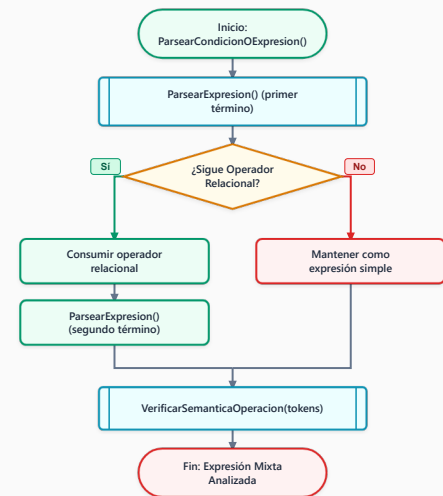


DIAGRAMA DE FLUJO: PARSEARCONDICIONOEXPRESION()

ParsearExpresion()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearExpresion()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Valida expresiones compuestas por términos enlazados mediante operadores de adición y sustracción (+ y -).

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Nivel 4 de la gramática de expresiones. Invoca primero a 'ParsearTermino()'. A continuación, mediante un bucle 'while', comprueba si el token actual es una suma o resta ('AO+', 'AO-', '+', '-'). Si coincide, avanza el cursor y llama nuevamente a 'ParsearTermino()', asegurando la precedencia de operadores.

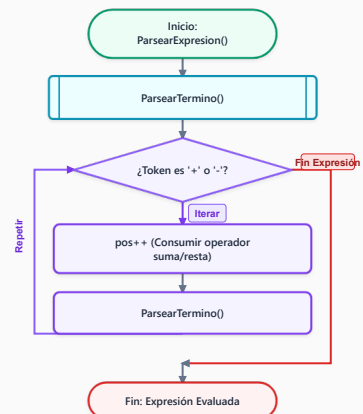


DIAGRAMA DE FLUJO: PARSEAREXPRESION()

ParsearTermino()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearTermino()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Valida términos compuestos por factores de mayor jerarquía vinculados mediante multiplicación, división o módulo (*, /, %).

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Nivel 3 de la gramática. Llama a 'ParsearPotencia()'. Mediante un bucle 'while', evalúa si el token actual es de multiplicación, división o módulo ('AO*', 'AO/', 'AO%', '*', '/', '%'). Consume el operador y procesa el siguiente factor mediante 'ParsearPotencia()', garantizando que la multiplicación y división se agrupen antes de la suma y la resta.

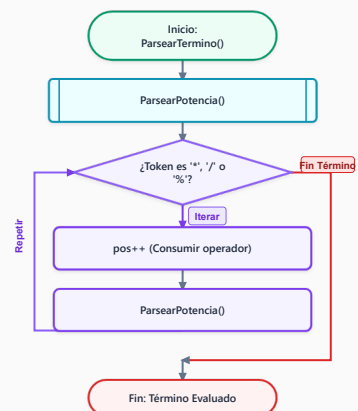


DIAGRAMA DE FLUJO: PARSEARTERMINO()

ParsearPotencia()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearPotencia()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Valida operaciones de exponenciación o potencia respetando la jerarquía matemática (operador ^).

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Nivel 2 de la gramática. Invoca primero a 'ParsearFactor()'. En un bucle 'while', verifica si el token corresponde al operador de potencia ('AO^', 'SC^', '^'). Consume el operador y llama a 'ParsearFactor()' para la base o exponente subsiguiente.

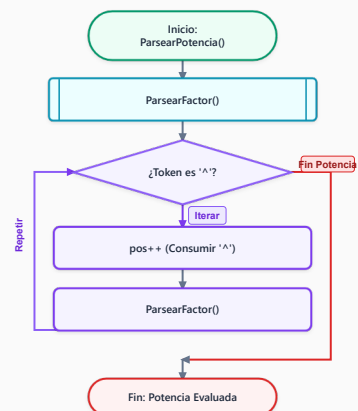


DIAGRAMA DE FLUJO: PARSEARPOTENCIA()

ParsearFactor()

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ParsearFactor()
```

PROPÓSITO Y ROL EN EL SISTEMA

Analiza los factores atómicos de la expresión: constantes, identificadores, signos unarios o subexpresiones entre paréntesis.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Comprueba si existe un signo unario '+' o '-' y lo consume permitiendo literales negativos o positivos con signo explícito.
2. Si el token es un identificador ('IDV') o una constante ('INC', 'RNC', 'NUC', 'STR'), lo acepta consumiendo el token.
3. Si encuentra un paréntesis de apertura '(' ('SC('), lo consume, invoca recursivamente a 'ParsearExpresion()' y exige el paréntesis de cierre ')' ('SC)').
4. Si no coincide con ninguno de los operandos válidos, emite un error sintáctico con 'ReportarError' y avanza el cursor para no ciclarse.

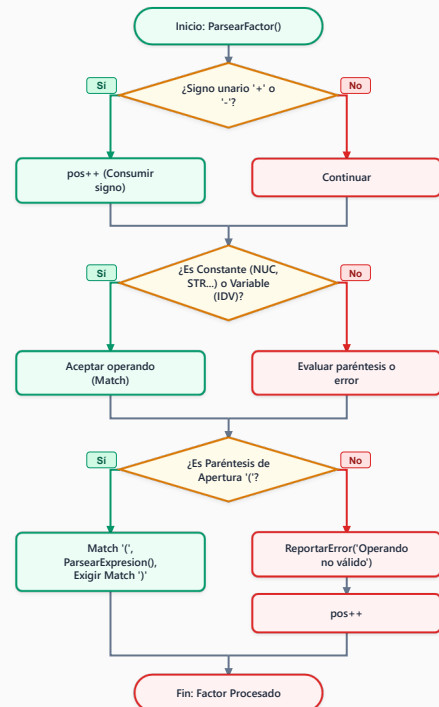


DIAGRAMA DE FLUJO: PARSEARFACTOR()

VerificarSemanticaOperacion(List<TokenSintactico> tokensOp)

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void VerificarSemanticaOperacion(List<TokenSintactico> tokensOp)
```

PROPÓSITO Y ROL EN EL SISTEMA

Conecta el analizador sintáctico con el semántico, transfiriendo los tokens de una expresión para inferir tipos y validar compatibilidad.

PARÁMETROS Y RETORNO

Parámetros: List<TokenSintactico> tokensOp: Secuencia de tokens correspondiente a la expresión u operación

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si 'tokensOp' es nulo o vacío, retorna sin acción.
2. Invoca a 'semantico.AnalizarTokensCompilador(tokensOp)'.
3. Añade el resultado devuelto a la lista 'OperacionesDetectadas'.
4. Si el resultado contiene errores semánticos ('ErroresSemanticos'), itera sobre cada uno de ellos extrayendo la línea y llamando a 'ReportarErrorSemantico(err, linea)' para mostrarlos en color púrpura en la tabla de errores.

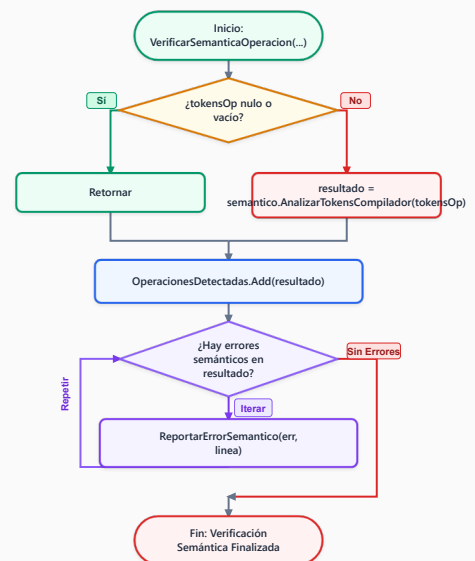


DIAGRAMA DE FLUJO: VERIFICARSEMANTICAOPERACION(LIST<TOKENSINTACTICO> TOKENSOP)

Match(params string[] esperados)

PRIVATE

BOOL (TRUE SI COINCIDIÓ Y CONSUMIÓ EL TOKEN, FALSE SI NO)

ANALIZADORSINTACTICO.CS

```
private bool Match(params string[] esperados)
```

PROPÓSITO Y ROL EN EL SISTEMA

Comprueba si el token actual coincide por prefijo de tipo o coincidencia exacta de lexema con alguna de las opciones esperadas y avanza el cursor.

PARÁMETROS Y RETORNO

Parámetros: params string[] esperados: Lista de tipos o lexemas válidos

Retorno: bool (true si coincidió y consumió el token, false si no)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si `pos >= tokens.Count`, retorna `false`.
2. Itera sobre cada cadena en `esperados`.
3. Si `tipoActual.StartsWith(exp)` o `lexemaActual.Equals(exp, OrdinalIgnoreCase)`, incrementa `pos++` y retorna `true`.
4. Si ninguna opción coincide tras revisar toda la lista, retorna `false` sin mover la posición del cursor.

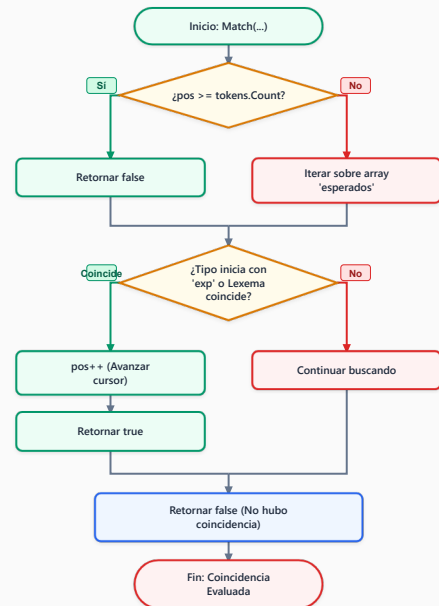


DIAGRAMA DE FLUJO: MATCH(PARAMS STRING[] ESPERADOS)

ReportarError(string mensaje)

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ReportarError(string mensaje)
```

PROPÓSITO Y ROL EN EL SISTEMA

Registra un error sintáctico con su número de línea en el DataGridView de la interfaz y lo resalta visualmente en color naranja.

PARÁMETROS Y RETORNO

Parámetros: string mensaje: Descripción detallada del error sintáctico

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Extrae el número de línea del token en la posición actual `tokens[pos].Linea` o del último token analizado.
2. Añade una nueva fila al control `dgtErrores` con la línea y el mensaje prefijado con 'Error Sintáctico: '.
3. Configura el color de fondo de la celda en `Color.Orange` para diferenciar inmediatamente los errores sintácticos de los léxicos o semánticos.

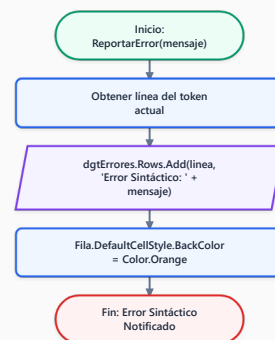


DIAGRAMA DE FLUJO: REPORTARERROR(STRING MENSAJE)

ReportarErrorSemantico(string mensaje, int linea)

PRIVATE

VOID

ANALIZADORSINTACTICO.CS

```
private void ReportarErrorSemantico(string mensaje, int linea)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Registra un error semántico en el DataGridView de la interfaz gráfica y lo resalta con color morado distintivo.

🔗 PARÁMETROS Y RETORNO

Parámetros: string mensaje: Descripción del error de tipos o semántica; int linea:

Línea de código fuente

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Agrega una nueva fila a `dgtErrores` con el número de línea y el mensaje semántico recibido.
2. Establece `DefaultCellStyle.BackColor = Color.MediumPurple` en la última fila agregada, facilitando al programador identificar fallas como incompatibilidad de tipos en operaciones o variables no declaradas.

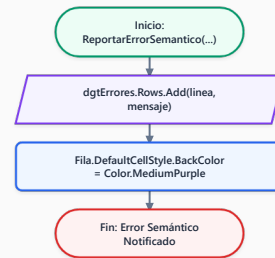


DIAGRAMA DE FLUJO: REPORTARERRORESEMANTICO(STRING MENSAJE, INT LINEA)

Módulo 7: Analizador Semántico, Inferencia y Árbol de Jerarquía

(AnalizadorSemanticoJerarquia.cs)

Motor de análisis semántico y jerarquía de operadores matemáticos y lógicos. Tokeniza expresiones, valida el equilibrio de paréntesis, construye el árbol jerárquico según los 7 niveles de precedencia, infiere tipos de datos con promoción automática, evalúa constantes y genera los pasos explicativos de resolución.

AnalizadorSemanticoJerarquia(...) [Constructor]

PUBLIC

[CONSTRUCTOR]

ANALIZADORSEMANTICOJERARQUIA.CS

```
public AnalizadorSemanticoJerarquia(Dictionary<string, string>? variablesConTipo = null)
```

PROPÓSITO Y ROL EN EL SISTEMA

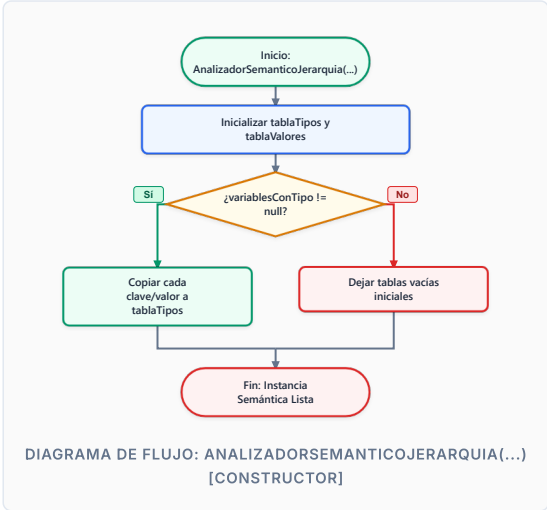
Instancia el analizador semántico e inicializa las tablas de tipos y de valores calculados con insensibilidad a mayúsculas/minúsculas.

PARÁMETROS Y RETORNO

Parámetros: Dictionary<string, string>? variablesConTipo: Diccionario opcional con variables y tipos preexistentes
Retorno: [Constructor]

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Crea internamente 'tablaTipos' y 'tablaValores' como diccionarios que usan 'StringComparer.OrdinalIgnoreCase'. Si se le pasa una colección inicial de variables con tipo, las copia una a una en 'tablaTipos'. Prepara las estructuras auxiliares de pasos de evaluación, lista de tokens y recolector de errores semánticos.



RegistrarVariable(string nombre, string tipoDato, string? valor = null)

PUBLIC

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
public void RegistrarVariable(string nombre, string tipoDato, string? valor = null)
```

PROPÓSITO Y ROL EN EL SISTEMA

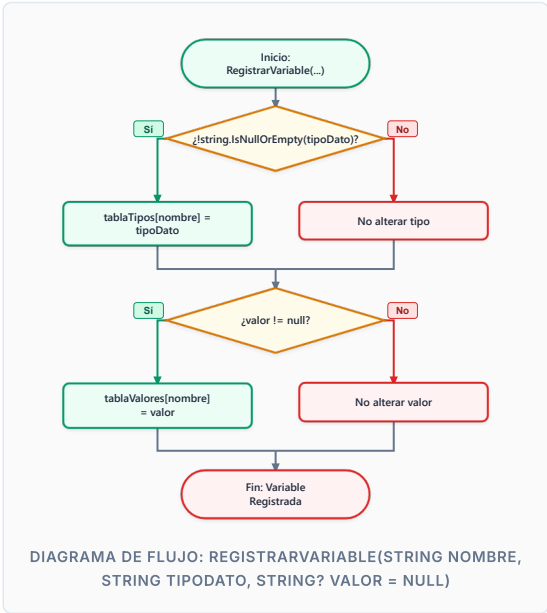
Registra o actualiza de manera persistente el tipo de dato y valor conocido de una variable en la tabla de símbolos semántica.

PARÁMETROS Y RETORNO

Parámetros: nombre: Identificador de la variable; tipoDato: Tipo de dato (int, float, string, bool); valor: Valor opcional calculado
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- Si 'tipoDato' no es nulo ni vacío, asigna 'tablaTipos[nombre] = tipoDato'.
 - Si 'valor' no es nulo, actualiza 'tablaValores[nombre] = valor'.
- Esta información se utiliza posteriormente para la inferencia de tipos durante la resolución del árbol jerárquico y para la propagación de constantes.



TieneVariable(string nombre)

PUBLIC

BOOL (TRUE SI LA VARIABLE ESTÁ REGISTRADA CON UN TIPO VÁLIDO)

ANALIZADORSEMANTICOJERARQUIA.CS

```
public bool TieneVariable(string nombre)
```

PROPÓSITO Y ROL EN EL SISTEMA

Verifica si una variable ha sido declarada previamente y posee un tipo de dato asignado diferente de nulo.

PARÁMETROS Y RETORNO

Parámetros: string nombre: Nombre o identificador de la variable a consultar
Retorno: bool (true si la variable está registrada con un tipo válido)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Consulta si `tablaTipos.ContainsKey(nombre)` es verdadero, verificando adicionalmente que el valor no sea nulo, ni vacío, ni la cadena 'Null'. Retorna `true` si la variable está lista para ser usada en expresiones, previniendo errores de variables no declaradas.

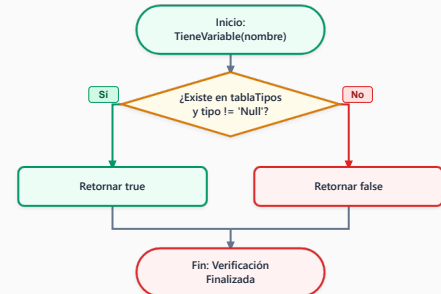


DIAGRAMA DE FLUJO: TIENEVARIABLE(STRING NOMBRE)

ObtenerTipo(string nombre) y ObtenerValor(string nombre)

PUBLIC

STRING (TIPO DE DATO) / STRING?
(VALOR TEXTUAL O NULL)

ANALIZADORSEMANTICOJERARQUIA.CS

```
public string ObtenerTipo(string nombre)
public string? ObtenerValor(string nombre)
```

PROPÓSITO Y ROL EN EL SISTEMA

Métodos públicos de consulta para extraer el tipo de dato y el valor calculado actual de una variable registrada.

PARÁMETROS Y RETORNO

Parámetros: string nombre: Nombre del identificador
Retorno: string (Tipo de dato) / string? (Valor textual o null)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Delegan internamente a `ObtenerTipoVariable(nombre)` y `ObtenerValorVariable(nombre)` respectivamente. Permiten al formulario principal y a otros componentes sincronizar las celdas de la tabla de símbolos en el DataGridView con los valores semánticos más recientes.

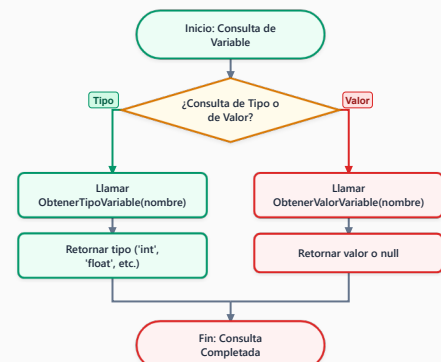


DIAGRAMA DE FLUJO: OBTENERTIPO(STRING NOMBRE) Y OBTENERVALOR(STRING NOMBRE)

ObtenerTablaTipos() y ObtenerTablaValores()

PUBLIC

Dictionary<String, String> (COPIA
DEFENSIVA DE LA TABLA)

ANALIZADORSEMANTICOJERARQUIA.CS

```
public Dictionary<string, string> ObtenerTablaTipos()  
public Dictionary<string, string> ObtenerTablaValores()
```

PROPÓSITO Y ROL EN EL SISTEMA

Retorna copias completas e independientes de los diccionarios de tipos y valores para evitar mutaciones accidentales del estado interno.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: Dictionary<string, string> (Copia defensiva de la tabla)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Crea una nueva instancia de 'Dictionary<string, string>' pasando como fuente la tabla interna correspondiente y el comparador 'OrdinalIgnoreCase'. Garantiza la encapsulación de los datos del analizador semántico.

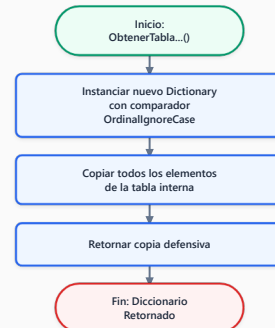


DIAGRAMA DE FLUJO: OBTENERTABLATIPOS() Y
OBTENERTABLAVALORES()

AnalizarExpresion(string expresionTexto, int linea = 1)

PUBLIC

ResultadoSemanticoOperacion

ANALIZADORSEMANTICOJERARQUIA.CS

```
public ResultadoSemanticoOperacion AnalizarExpresion(string expresionTexto, int linea = 1)
```

PROPÓSITO Y ROL EN EL SISTEMA

Función principal de análisis semántico que tokeniza una expresión, construye su árbol de jerarquía, infiere tipos, evalúa y genera pasos.

PARÁMETROS Y RETORNO

Parámetros: expresionTexto: Cadena con la expresión matemática/lógica; linea: Número de línea en el código fuente

Retorno: ResultadoSemanticoOperacion

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Limpia las listas internas de errores y pasos.
2. Convierte el texto a tokens semánticos mediante 'Tokenizar'.
3. Si no hay tokens, retorna indicando operación vacía.
4. Invoca 'VerificarBalanceParentesis()' para detectar desequilibrios en '(' y ')'.
5. Construye el árbol sintáctico abstracto llamando a 'ParsearAsignacion()' (raíz de la jerarquía).
6. Si hay tokens sobrantes al final, genera error de token inesperado.
7. Si el árbol raíz se construyó, ejecuta 'InferirTiposYValores(raiz)' para propagar tipos y evaluar valores constantes.
8. Genera los pasos ordenados de resolución con 'GenerarPasosJerarquia(raiz)'.
9. Empaqueta el árbol, pasos, errores y tipo final en un objeto 'ResultadoSemanticoOperacion' y lo retorna.

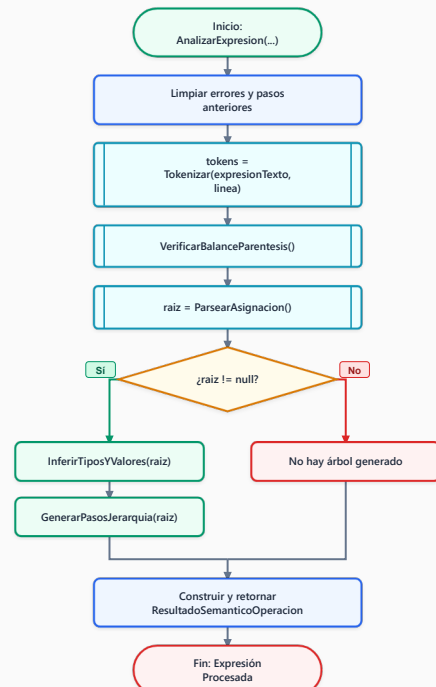


DIAGRAMA DE FLUJO: ANALIZAREXPRESION(STRING
EXPRESIONTEXTO, INT LINEA = 1)

AnalizarTokensCompilador(List<TokenSintactico> tokensSintacticos)

PUBLIC

RESULTADOSEMANTICOOPERACION

ANALIZADORSEMANTICOJERARQUIA.CS

```
public ResultadoSemanticoOperacion AnalizarTokensCompilador(List<TokenSintactico> tokensSintacticos)
```

PROPÓSITO Y ROL EN EL SISTEMA

Convierte una lista de tokens sintácticos en una representación textual normalizada y delega su análisis a AnalizarExpresion.

PARÁMETROS Y RETORNO

Parámetros: List<TokenSintactico> tokensSintacticos: Lista de tokens provenientes del analizador sintáctico

Retorno: ResultadoSemanticoOperacion

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Itera sobre 'tokensSintacticos' utilizando un 'StringBuilder'.
2. Normaliza símbolos de operadores: sustituye códigos léxicos como 'ALO=' por '=', 'AO+' por '+', 'AO*' por '*', etc.
3. Reconstruye la cadena completa de la expresión preservando espacios.
4. Invoca 'AnalizarExpresion(sb.ToString(), linea)' y devuelve el resultado evaluado.

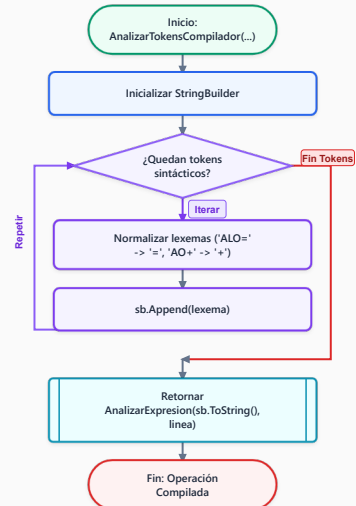


DIAGRAMA DE FLUJO:
ANALIZARTOKENSCOMPILADOR(LIST<TOKENSINTACTICO>
TOKENSSINTACTICOS)

Tokenizar(string entrada, int linea)

PRIVATE

LIST<TOKENSEMANTICO>

ANALIZADORSEMANTICOJERARQUIA.CS

```
private List<TokenSemantico> Tokenizar(string entrada, int linea)
```

PROPÓSITO Y ROL EN EL SISTEMA

Descompone el texto de una expresión en una secuencia de tokens semánticos clasificados léxicamente.

PARÁMETROS Y RETORNO

Parámetros: string entrada: Expresión como texto sin procesar; int linea: Línea de procedencia

Retorno: List<TokenSemantico>

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Recorre carácter por carácter la cadena:

1. Ignora espacios en blanco.
2. Si encuentra comillas (''), lee la cadena literal completa hasta la comilla de cierre y crea un token 'STR'.
3. Si encuentra dígitos, analiza el número entero o real con punto decimal ('NUM').
4. Si encuentra letras, lee identificadores o palabras reservadas de operadores lógicos como 'AND', 'OR', 'NOT' ('ID' u 'OP_LOG').
5. Reconoce operadores de dos caracteres ('==', '!=', '<=', '>=', '&&', '||').
6. Reconoce operadores aritméticos individuales ('+', '-', '*', '/', '^', '%'), paréntesis ('(', ')') y asignación ('=').
7. Retorna la lista resultante de 'TokenSemantico'.

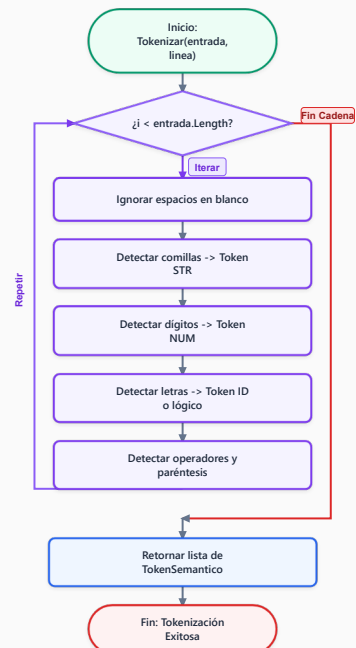


DIAGRAMA DE FLUJO: TOKENIZAR(STRING ENTRADA, INT
LINEA)

VerificarBalanceParentesis()

PRIVATE

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
private void VerificarBalanceParentesis()
```

PROPÓSITO Y ROL EN EL SISTEMA

Realiza un análisis estático preliminar para garantizar que cada paréntesis de apertura tenga su correspondiente cierre ordenado.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Utiliza una pila conceptual o contador de balance ('abiertos' = 0). Recorre los tokens semánticos: incrementa el contador al hallar 'PAR_ABRE' '(' y lo decrementa al hallar 'PAR_CIERRA' ')'. Si el contador se vuelve negativo en algún punto, reporta que hay un paréntesis de cierre sin apertura. Al final del recorrido, si el contador es mayor que cero, reporta que quedaron paréntesis abiertos sin cerrar.

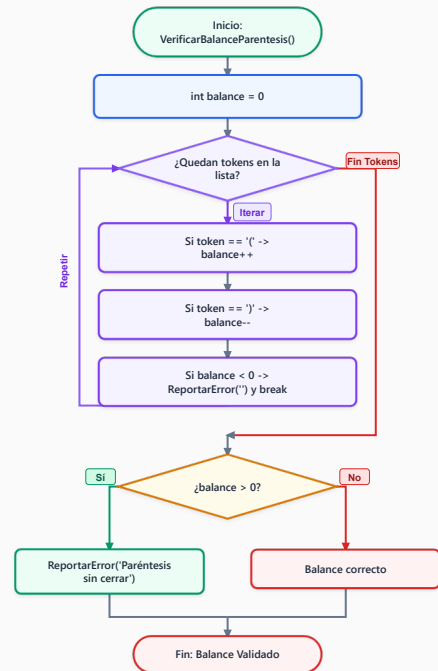


DIAGRAMA DE FLUJO: VERIFICARBALANCEPARENTESIS()

ParsearAsignacion()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearAsignacion()
```

PROPÓSITO Y ROL EN EL SISTEMA

Cúspide de la jerarquía (Nivel 7). Analiza sentencias de asignación donde una variable recibe el resultado de una expresión.

PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: NodoJerarquia

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Llama a 'ParsearLogico()' para procesar la subexpresión.
2. Si el token actual es un identificador y el siguiente token es el operador de asignación '=', crea un nodo raíz de asignación de 'NivelJerarquia = 7'.
3. Asigna la variable como nodo hijo izquierdo.
4. Consume el signo '=' y llama recursivamente a 'ParsearAsignacion()' o 'ParsearLogico()' para el hijo derecho.
5. Si no es una asignación, retorna directamente el subárbol lógico obtenido.

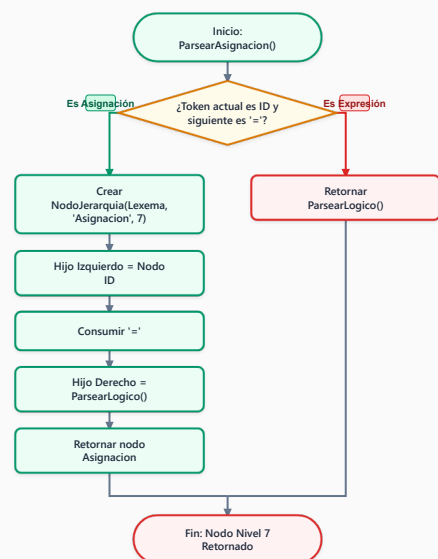


DIAGRAMA DE FLUJO: PARSEARASIGNACION()

ParsearLogico()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearLogico()
```

🎯 PROPÓSITO Y ROL EN EL SISTEMA

Nivel 6 de la jerarquía. Analiza operaciones lógicas binarias de conjunción y disyunción (AND, OR, &&, ||).

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: NodoJerarquia

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Procesa el operando izquierdo mediante 'ParsearRelacional()'.
2. En un ciclo 'while', verifica si el token actual es un operador lógico ('AND', 'OR', '&&', '||').
3. Si coincide, consume el token, crea un 'NodoJerarquia' con 'NivelJerarquia = 6', asigna el subárbol izquierdo existente y procesa el subárbol derecho con 'ParsearRelacional()'.
4. Actualiza la raíz con el nuevo nodo y continúa mientras existan operadores lógicos encadenados.

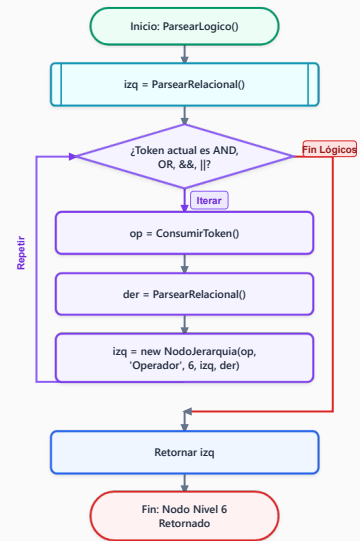


DIAGRAMA DE FLUJO: PARSEARLOGICO()

ParsearRelacional()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearRelacional()
```

🎯 PROPÓSITO Y ROL EN EL SISTEMA

Nivel 5 de la jerarquía. Analiza operadores relacionales y de comparación (==, !=, <, >, <=, >=).

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: NodoJerarquia

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Procesa el operando izquierdo invocando 'ParsearSumaResta()'.
2. En un bucle 'while', comprueba si el token actual es un operador relacional.
3. Crea un 'NodoJerarquia' con 'NivelJerarquia = 5' y descripción 'Operación Relacional', vinculando el lado izquierdo y analizando el lado derecho mediante 'ParsearSumaResta()'.
4. Retorna el nodo resultante que producirá un valor de tipo booleano ('bool').

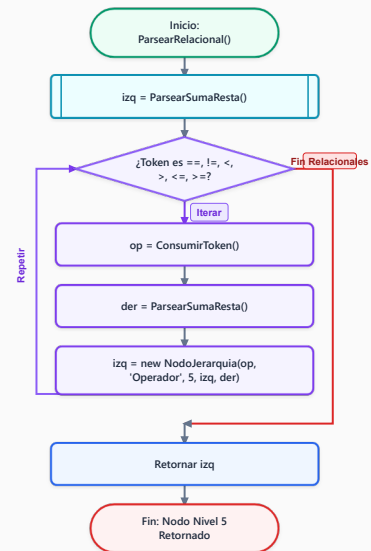


DIAGRAMA DE FLUJO: PARSEARRELACIONAL()

ParsearSumaResta()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearSumaResta()
```

PROPÓSITO Y ROL EN EL SISTEMA

Nivel 4 de la jerarquía. Analiza operaciones aritméticas de adición y sustracción (+ y -).

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: NodoJerarquia

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Llama a 'ParsearMultiplicacionDivision()' para obtener el operando izquierdo.
2. Mientras el token sea '+', '-', consume el operador.
3. Crea un 'NodoJerarquia' con 'NivelJerarquia = 4' y descripción 'Adición' o 'Sustracción'.
4. Enlaza 'izq' y obtiene 'der' con 'ParsearMultiplicacionDivision()', asegurando asociatividad por la izquierda.

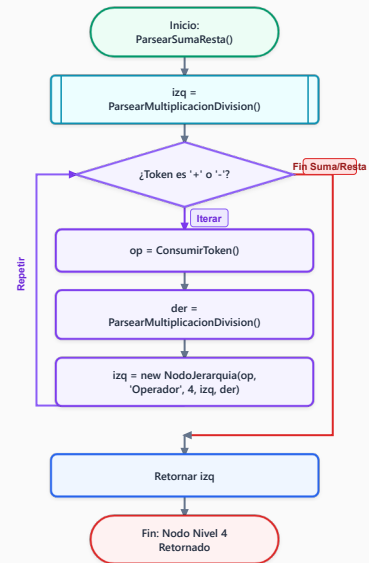


DIAGRAMA DE FLUJO: PARSEARSUMARESTA()

ParsearMultiplicacionDivision()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearMultiplicacionDivision()
```

PROPÓSITO Y ROL EN EL SISTEMA

Nivel 3 de la jerarquía. Analiza operaciones aritméticas de multiplicación, división y módulo (*, /, %).

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: NodoJerarquia

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Obtiene el operando izquierdo con 'ParsearPotencia()'.
2. En un ciclo 'while', comprueba si el token es '*', '/', o '%'.
3. Crea un 'NodoJerarquia' con 'NivelJerarquia = 3' ('Multiplicación', 'División' o 'Módulo').
4. Enlaza los hijos y resuelve el subárbol derecho con 'ParsearPotencia()'.
5. Retorna la raíz acumulada.

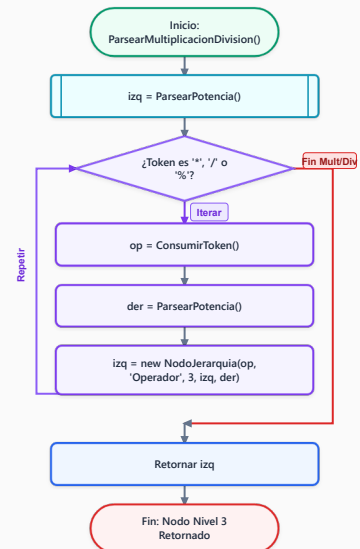


DIAGRAMA DE FLUJO: PARSEARMULTIPLICACIONDIVISION()

ParsearPotencia()

PRIVATE

NODOJERARQUIA

ANALIZADORSEMANTICOJERARQUIA.CS

```
private NodoJerarquia ParsearPotencia()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Nivel 2 de la jerarquía. Procesa operaciones matemáticas de potenciación con el operador ^.

🔧 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: NodoJerarquia

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Obtiene el factor base llamando a 'ParsearPrimario()'.
2. Mientras el token sea '^', lo consume y crea un 'NodoJerarquia' con 'NivelJerarquia = 2' ('Potencia').
3. Enlaza la base y resuelve el exponente con 'ParsearPotencia()' (asociatividad por la derecha) o 'ParsearPrimario()'.
4. Retorna el nodo de potencia.

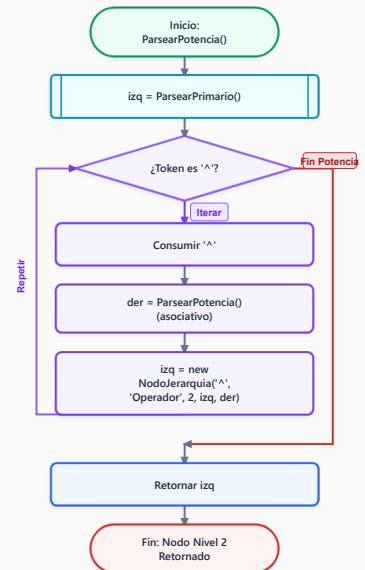


DIAGRAMA DE FLUJO: PARSEARPOTENCIA()

```
private NodoJerarquia ParsearPrimario()
```

PROPÓSITO Y ROL EN EL SISTEMA

Nivel 1 de la jerarquía. Maneja paréntesis con prioridad máxima, literales constantes, identificadores y signos unarios.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: NodoJerarquia

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si encuentra un signo unario '+' o '-', consume el signo y crea un nodo de signo unario o lo adjunta a la constante siguiente.
2. Si encuentra paréntesis '(', consume el token, crea un nodo contenedor con 'NivelJerarquia = 1' ('Agrupación (Prioridad Alta)'), evalúa la subexpresión interna llamando a 'ParsearLogico()' y exige el paréntesis de cierre ')'. Retornar nodo
3. Si es una constante numérica ('NUM'), infiere si es 'int' o 'float' y crea un nodo de constante.
4. Si es una cadena literal ('STR'), crea un nodo de constante de tipo 'string'.
5. Si es un identificador ('ID'), consulta su tipo en la tabla de símbolos y crea un nodo identificador.
6. Si no coincide con ningún caso, reporta error semántico y retorna un nodo de error.

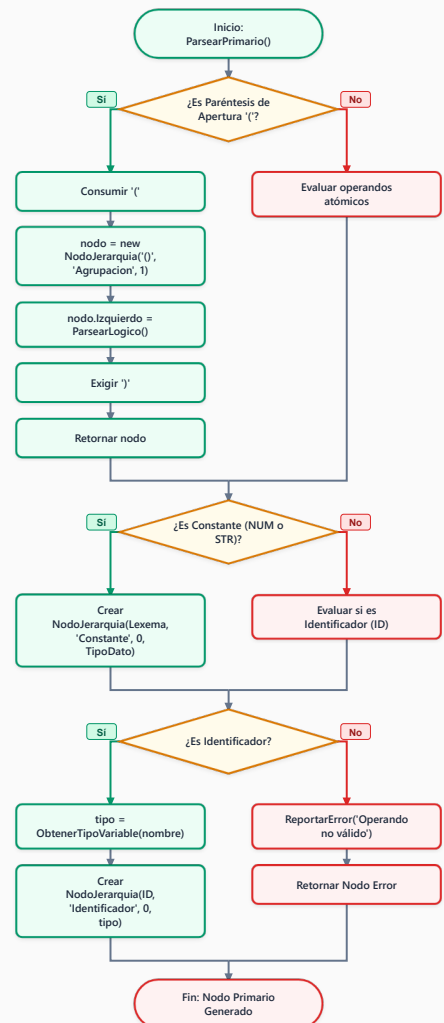


DIAGRAMA DE FLUJO: PARSEARPRIMARIO()

ObtenerTipoVariable(string nombreVar) y ObtenerValorVariable(string nombreVar)

PRIVATE

STRING
/
STRING?

ANALIZADOR SEMANTICO JERARQUIA.CS

```
private string ObtenerTipoVariable(string nombreVar)  
private string? ObtenerValorVariable(string nombreVar)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Consulta de forma segura y recupera el tipo y valor asignado de una variable, o reporta si la variable es desconocida.

🔗 PARÁMETROS Y RETORNO

Parámetros: string nombreVar: Nombre de la variable a consultar en la tabla de símbolos

Retorno: string / string?

🔗 EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- 'ObtenerTipoVariable': Si 'tablaTipos.ContainsKey(nombreVar)' y no es 'Null', retorna el tipo registrado ('int', 'float', 'string', 'bool'). Si no existe, reporta error semántico 'Variable no declarada o sin tipo asignado' y retorna 'desconocido'.
- 'ObtenerValorVariable': Retorna 'tablaValores[nombreVar]' si existe, o 'null' si aún no tiene un valor numérico/textual asignado.

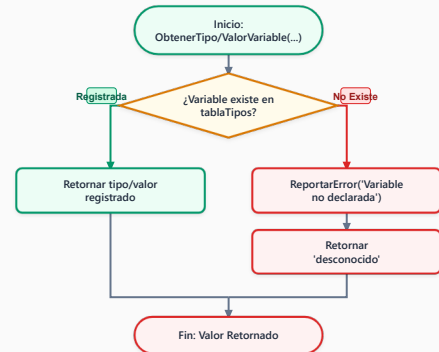


DIAGRAMA DE FLUJO: OBTENERTIPOVARIABLE(STRING NOMBREVAR) Y OBTENERVALORVARIABLE(STRING NOMBREVAR)

```
private void InferirTiposYValores(NodoJerarquia nodo)
```

PROPÓSITO Y ROL EN EL SISTEMA

Realiza un recorrido post-orden recursivo para deducir los tipos de datos de cada subexpresión, evaluar operaciones constantes y verificar compatibilidad semántica.

PARÁMETROS Y RETORNO

Parámetros: NodoJerarquia nodo: Nodo actual del árbol que se está evaluando
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si el nodo es nulo, termina.
2. Recorre recursivamente 'Izquierdo' y 'Derecho' en post-orden.
3. Si es un nodo de agrupación (paréntesis), hereda directamente el tipo y valor de su hijo izquierdo.
4. Si es operador aritmético (+, -, *, /, ^, %):
 - Verifica que los operandos sean numéricos (int o float) o cadenas compatibles con concatenación (+).
 - Si un operando es float y otro int, promociona el tipo resultante a 'float'.
 - Si ambos son int, el tipo es 'int' (excepto división real si aplica).
 - Valida división entre cero ('valDer == 0'). De ocurrir, marca 'nodo.TieneError = true' y reporta error semántico.
 - Si ambos operandos tienen valores constantes calculados, efectúa el cómputo matemático exacto y guarda el resultado en 'ValorCalculado'.
5. Si es operador relacional (==, <, >, etc.) o lógico (AND, OR), verifica la compatibilidad de tipos y fija el tipo resultante en 'bool'.
6. Si es asignación ('='), valida que el tipo del valor derecho sea compatible con la variable receptora y actualiza las tablas semánticas.

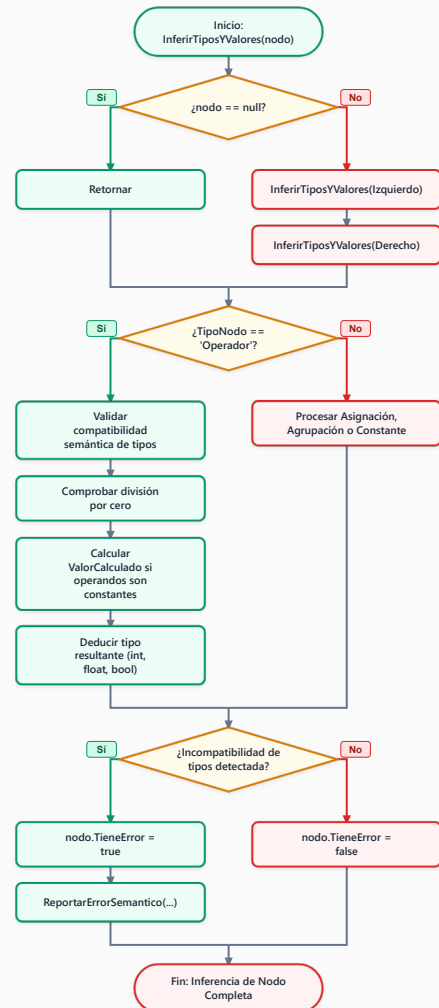


DIAGRAMA DE FLUJO:
INFERIRTIPOSYVALORES(NODOJERARQUIA NODO)

FormatearNumeroConSigno(double valor, bool esEntero)

PRIVATE

STRING (NÚMERO FORMATEADO)

ANALIZADORSEMANTICOJERARQUIA.CS

```
private string FormatearNumeroConSigno(double valor, bool esEntero)
```

PROPÓSITO Y ROL EN EL SISTEMA

Formatea valores numéricos con punto decimal invariante (CultureInfo.InvariantCulture) evitando discrepancias por configuraciones regionales del sistema operativo.

PARÁMETROS Y RETORNO

Parámetros: double valor: Valor numérico a formatear; bool esEntero: Si debe presentarse sin decimales

Retorno: string (Número formateado)

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si 'esEntero', redondea o castea a entero largo '(long)valor' y retorna su cadena.
2. Si es real, utiliza 'valor.ToString("0.###", CultureInfo.InvariantCulture)' para garantizar el punto decimal '.' y eliminar ceros innecesarios al final.

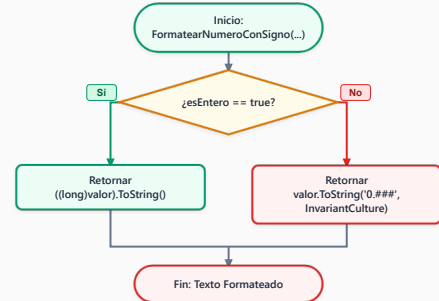


DIAGRAMA DE FLUJO: FORMATEARNUMEROCONSIGNO(DOUBLE VALOR, BOOL ESENTERO)

GenerarPasosJerarquia(NodoJerarquia nodo)

PRIVATE

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
private void GenerarPasosJerarquia(NodoJerarquia nodo)
```

PROPÓSITO Y ROL EN EL SISTEMA

Genera secuencialmente los pasos de resolución de la operación en estricto orden de prioridad jerárquica para mostrarlos en la interfaz gráfica.

PARÁMETROS Y RETORNO

Parámetros: NodoJerarquia nodo: Nodo del árbol a partir del cual se generan los pasos pedagógicos

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Realiza un recorrido post-orden del árbol (resuelve primero los hijos de mayor precedencia).
2. Si el nodo es una operación evaluable, genera una línea formateada con el número de paso 'Paso N: [Prioridad X - Tipo] Operación → Resultado'.
3. Añade la explicación textual a la lista interna 'pasos' que luego se visualiza en el RichTextBox de pasos jerárquicos.

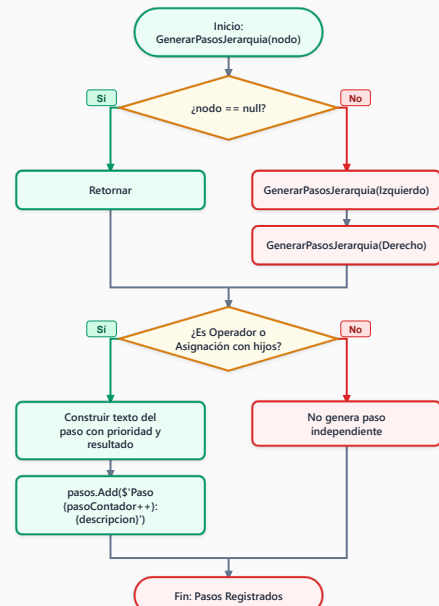


DIAGRAMA DE FLUJO: GENERARPASOSJERARQUIA(NODOJERARQUIA NODO)

ReportarError(string mensaje, int linea = 1)

PRIVATE

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
private void ReportarError(string mensaje, int linea = 1)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Almacena un mensaje de error semántico con formato normalizado en la lista de errores del analizador.

👤 PARÁMETROS Y RETORNO

Parámetros: string mensaje: Detalle del error; int linea: Línea del código fuente
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Formatea la cadena incluyendo el prefijo 'Línea {linea}: Error Semántico: {mensaje}' y la agrega a la lista 'errores'. Esto permite que al finalizar el análisis, el método llamador pueda consultar si la operación fue válida.



DIAGRAMA DE FLUJO: REPORTARERROR(STRING MENSAJE, INT LINEA = 1)

CargarArbolEnTreeView(TreeView tv, NodoJerarquia? raiz, string tituloOperacion)

PUBLIC

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
public void CargarArbolEnTreeView(TreeView tv, NodoJerarquia? raiz, string tituloOperacion)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Dibuja visualmente un árbol de expresión en el TreeView de la interfaz, expandiendo automáticamente todos sus niveles.

👤 PARÁMETROS Y RETORNO

Parámetros: tv: Control Windows Forms TreeView; raiz: Nodo raíz del árbol semántico; tituloOperacion: Título del nodo principal
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- Utiliza 'tv.BeginUpdate()' para pausar el repintado y mejorar el rendimiento gráfico.
- Limpia los nodos existentes en 'tv.Nodes'.
- Si 'raiz == null', crea un nodo informativo indicando que no hay árbol.
- Si 'raiz' existe, crea un nodo principal con 'tituloOperacion', añade el subárbol generado por 'raiz.ToTreeNode()' y llama a 'tv.ExpandAll()'.
- Restablece el repintado con 'tv.EndUpdate()'.

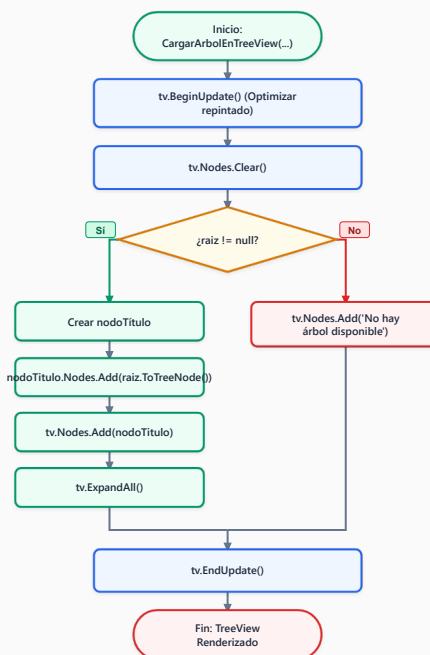


DIAGRAMA DE FLUJO: CARGARARBOLENTREEVIEW(TREEVIEW TV, NODOJERARQUIA? RAIZ, STRING TITULOOPERACION)

ExtraerOperacionesDeCodigoFuente(string codigoFuente)

PUBLIC

LIST<RESULTADOSEMANTICOOPERACION>

ANALIZADORSEMANTICOJERARQUIA.CS

```
public List<ResultadoSemanticoOperacion> ExtraerOperacionesDeCodigoFuente(string codigoFuente)
```

PROPÓSITO Y ROL EN EL SISTEMA

Examina el código fuente línea por línea para identificar y compilar todas las expresiones aritméticas, lógicas y asignaciones presentes.

PARÁMETROS Y RETORNO

Parámetros: string codigoFuente: Código completo del programa fuente

Retorno: List<ResultadoSemanticoOperacion>

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Divide el texto por líneas.
2. Para cada línea, busca patrones de asignación ('variable = expresion'), sentencias condicionales ('IF', 'WHILE'), o llamadas 'PRINT'.
3. Extrae la subexpresión matemática o lógica correspondiente.
4. Invoca a 'AnalizarExpresion(expr, numLinea)'.
5. Agrega el resultado a una lista de operaciones y la retorna al formulario para poblar el ComboBox de operaciones.

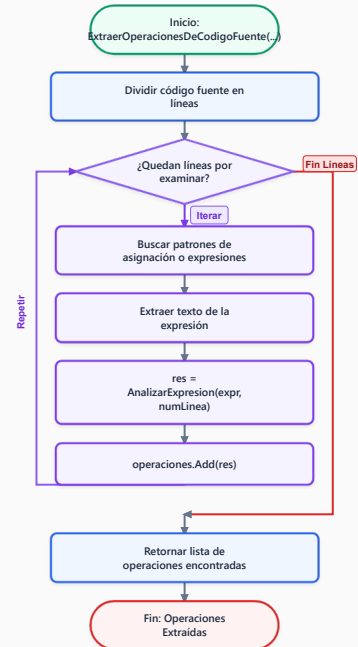


DIAGRAMA DE FLUJO:
EXTRAEROPERACIONESDECODIGOFUENTE(STRING
CODIGOFUENTE)

CargarArbolSincronizado(TreeView tv, List<ResultadoSemanticoOperacion> operaciones)

PUBLIC

VOID

ANALIZADORSEMANTICOJERARQUIA.CS

```
public void CargarArbolSincronizado(TreeView tv, List<ResultadoSemanticoOperacion> operaciones)
```

PROPÓSITO Y ROL EN EL SISTEMA

Carga todas las operaciones detectadas en el TreeView, agrupándolas como nodos principales para permitir su exploración interactiva.

PARÁMETROS Y RETORNO

Parámetros: tv: Control TreeView; operaciones: Lista de todas las operaciones semánticas analizadas

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Ejecuta `tv.BeginUpdate()`.
2. Limpia los nodos previos de `tv`.
3. Si `operaciones` está vacía, añade un nodo indicando que no hay operaciones.
4. Recorre cada operación detectada: crea un nodo con el icono de estado (✓/✗), asocia la instancia de la operación a la propiedad `Tag` del nodo, y añade su subárbol `ArbolRaiz.ToTreeNode()`.
5. Expande todos los nodos mediante `tv.ExpandAll()` y llama a `tv.EndUpdate()`.

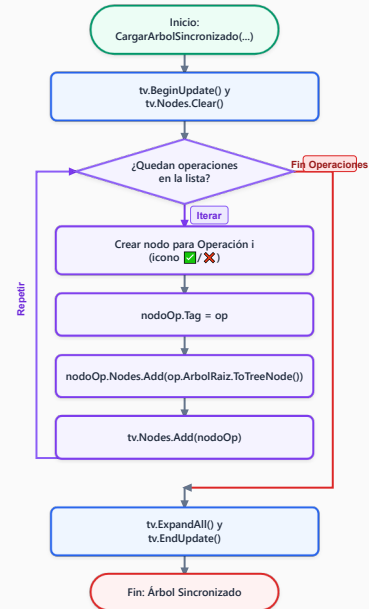


DIAGRAMA DE FLUJO: CARGARARBOLSINCRONIZADO(TREEVIEW TV, LIST<RESULTADOSEMANTICOOPERACION> OPERACIONES)

Módulo 8: Interfaz Gráfica de Usuario y Controlador Principal (Form1.cs)

Controlador visual central construido en Windows Forms. Conecta con SQL Server para recuperar la matriz de autómatas léxicos, coordina el pipeline de compilación de las 3 fases (Léxico, Sintáctico, Semántico), dibuja la numeración dinámica de líneas y sincroniza de forma interactiva el árbol de jerarquía con los pasos de resolución.

Form1() [Constructor]

PUBLIC [CONSTRUCTOR] FORM1.CS

```
public Form1()
```

PROPÓSITO Y ROL EN EL SISTEMA

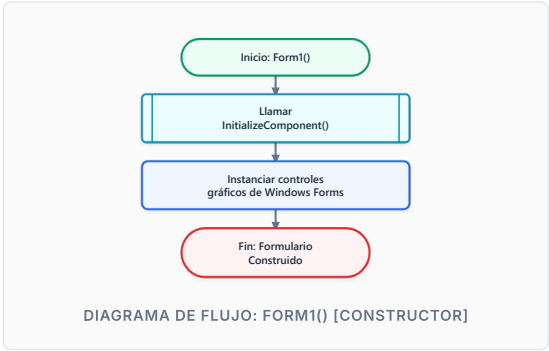
Inicializa los componentes gráficos del formulario Windows Forms definidos por el diseñador.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: [Constructor]

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Llama al método autogenerated `InitializeComponent()`, instanciando todos los controles visuales del compilador: cuadros de texto enriquecidos (`richProgramaFuente`, `richArchivoDeTokens`), tablas de datos (`dgtTablaDeSimbolos`, `dgtErrores`), lienzo de árbol (`tvArbolJerarquia`), botones de acción y paneles de numeración de líneas.



Form1_Load(object sender, EventArgs e)

PRIVATE VOID FORM1.CS

```
private void Form1_Load(object sender, EventArgs e)
```

PROPÓSITO Y ROL EN EL SISTEMA

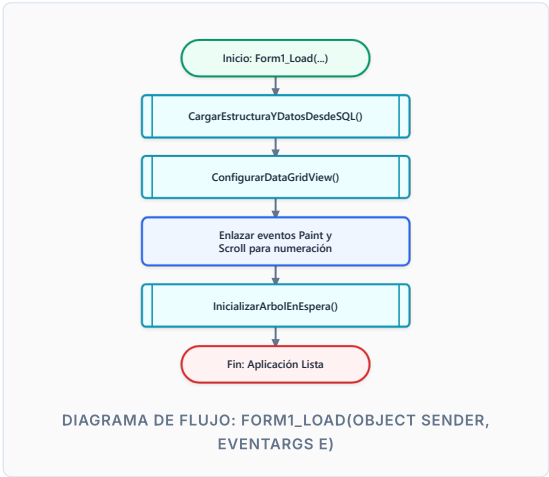
Configura el entorno inicial de la aplicación, conecta a SQL Server para obtener la matriz de transiciones y enlaza eventos de repintado.

PARÁMETROS Y RETORNO

Parámetros: object sender: Objeto emisor del evento; EventArgs e: Argumentos del evento de carga
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Invoca `CargarEstructuraYDatosDesdeSQL()` para cargar la matriz de autómatas finitos del analizador léxico desde SQL Server.
2. Llama a `ConfigurarDataGridView()` para definir el comportamiento de las tablas visuales.
3. Establece `richArchivoDeTokens.ReadOnly = true` para evitar edición manual de tokens.
4. Asocia los eventos de dibujo `picLineas\_Paint` y `picLinea2\_Paint` a los PictureBox de numeración de líneas.
5. Enlaza los eventos `VScroll` y `HScroll` de los editores para redibujar automáticamente las líneas al desplazarse.
6. Invoca `InicializarArbolEnEspera()` para presentar el estado inicial del árbol jerárquico.



CargarEstructuraYDatosDesdeSQL()

PRIVATE

VOID

FORM1.CS

```
private void CargarEstructuraYDatosDesdeSQL()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Establece la conexión con la base de datos SQL Server y carga la matriz de transiciones y alfabeto del autómata léxico.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Abre una conexión 'SqlConnection' utilizando 'ConexionBD'.
2. Ejecuta 'SELECT * FROM MatrizTransicion' rellenando un 'DataTable'.
3. Recorre las columnas de la tabla para registrar cada símbolo en 'alfabetoTemporal' (omitiendo columnas de control y ACEPTA).
4. Determina la cantidad máxima de estados y dimensiona la matriz en memoria 'matrizCompleta[totalFilas, totalColumnas]'.
5. Rellena las celdas de transición y las categorías de aceptación para cada estado.
6. Si la tabla está vacía o la conexión falla, emite una excepción descriptiva.

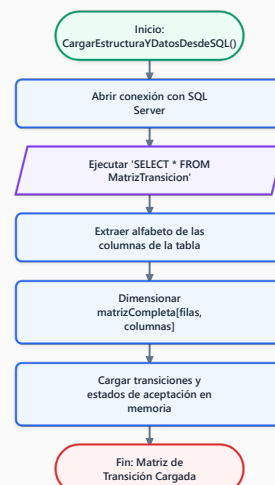


DIAGRAMA DE FLUJO:
CARGARESTRUCTURAYDATOSDESDESQ()

richProgramaFuente_TextChanged(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void richProgramaFuente_TextChanged(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Invalida el lienzo de numeración de líneas cuando el usuario teclea o edita código en el editor principal.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: Control RichTextBox; e: Argumentos del evento

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Invoca 'picLineas.Invalidate()', provocando que el sistema operativo vuelva a disparar el evento 'picLineas_Paint' para actualizar dinámicamente los números de línea visibles conforme el texto crece o disminuye.

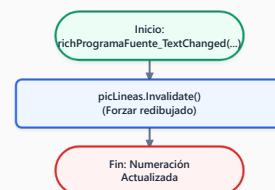


DIAGRAMA DE FLUJO:
RICHPROGRAMAFUENTE_TEXTCHANGED(OBJECT SENDER,
EVENTARGS E)

btnAnalizar_Click(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void btnAnalizar_Click(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Orquesta de principio a fin las 3 fases del compilador: Análisis Léxico, Análisis Sintáctico y Análisis Semántico con Árbol de Jerarquía.

👤 PARÁMETROS Y RETORNO

Parámetros: sender: Botón 'Analizar Todo'; e: Argumentos del evento

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Limpia controles gráficos: 'richArchivoDeTokens', 'dgtErrores', y vacía la lista de identificadores.
2. Fase Léxica: Itera sobre todas las líneas del código fuente invocando 'VerificarToken()' y presenta los tokens generados en 'richArchivoDeTokens'.
3. Preparación de Tokens: Convierte los tokens léxicos en instancias de 'TokenSintactico' con línea y lexema, filtrando tokens de error para evitar cascadas.
4. Registro de Variables: Pasa los identificadores detectados a 'analizadorSemantico'.
5. Fase Sintáctica y Semántica: Instancia 'AnalizadorSintactico' y llama a 'ParsearPrograma()', validando gramática y evaluando expresiones.
6. Compilación de Jerarquía: Llama a 'CompilarArbolJerarquia(sintactico)' para poblar el TreeView y los pasos de resolución.
7. Sincronización y Totales: Invoca 'ActualizarTablaDeSimbolos()' y coloca el conteo total de errores al final de la tabla.

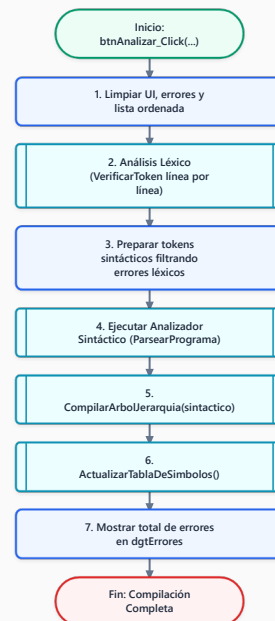


DIAGRAMA DE FLUJO: BTNANALIZAR_CLICK(OBJECT SENDER, EVENTARGS E)

AgregarErrores(string error, int linea)

PRIVATE

VOID

FORM1.CS

```
private void AgregarErrores(string error, int linea)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Traduce un código de error léxico a un mensaje comprensible, lo inserta en dgtErrores y lo resalta con fondo rojo.

👤 PARÁMETROS Y RETORNO

Parámetros: string error: Código interno del error léxico; int linea: Línea del archivo

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Busca la clave 'error' en el diccionario 'Errores2' (ej. '___EIDNV__' se traduce a 'Error identificador no valido').
2. Añade una fila en 'dgtErrores' con el número de línea ('linea + 1') y la descripción del error.
3. Configura 'DefaultCellStyle.BackColor = Color.Red' en la fila recién creada para alertar al usuario.

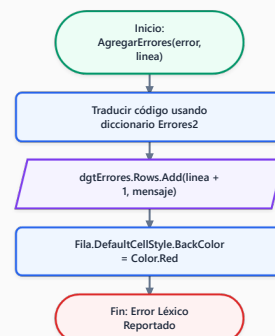


DIAGRAMA DE FLUJO: AGREGARERRORES(STRING ERROR, INT LINEA)

ConfigurarDataGridView()

PRIVATE

VOID

FORM1.CS

```
private void ConfigurarDataGridView()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Ajusta las propiedades de solo lectura, auto-ajuste de columnas y bloqueo de edición de las tablas de datos de la interfaz.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

🔗 EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Configura tanto 'dgtErrores' como 'dgtTablaDeSimbolos': deshabilita la adición y eliminación manual de filas por parte del usuario ('AllowUserToAddRows = false', 'AllowUserToDeleteRows = false'), activa el modo de solo lectura ('ReadOnly = true') y fija 'AutoSizeColumnsMode = Fill' para que las columnas se distribuyan proporcionalmente.

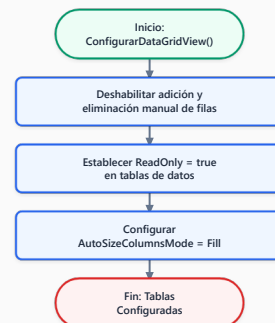


DIAGRAMA DE FLUJO: CONFIGURARDATAGRIDVIEW()

btnGuardarPrograma_Click(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void btnGuardarPrograma_Click(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Guarda el contenido del editor de código fuente en un archivo de texto en disco utilizando la clase Archivo.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: Botón de guardar; e: Argumentos del evento

Retorno: void

🔗 EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Instancia 'Archivo<string>' con la ruta predeterminada 'NombreArchivo'.
2. Si el archivo ya existía, lo elimina con 'EliminarArchivo()' para sobrescribirlo limpiamente.
3. Abre el archivo en modo escritura con 'HacerModoEscritura()'.
4. Itera sobre las líneas de 'richProgramaFuente' guardando cada una mediante 'AgregarObjeto(linea)'.
5. Cierra el archivo con 'CerrarArchivo()' y muestra un mensaje de confirmación.

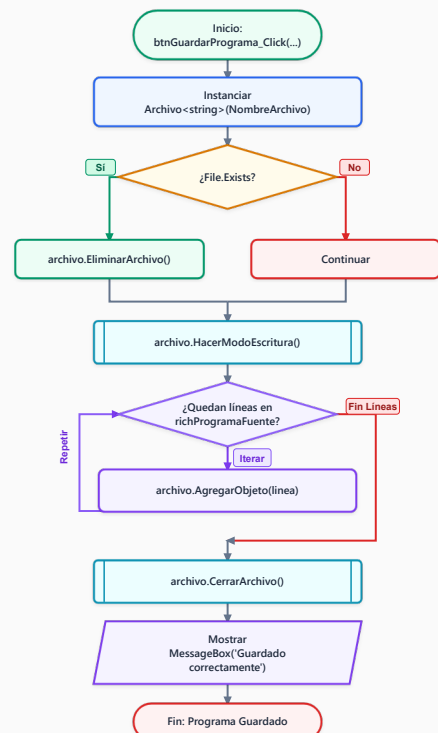


DIAGRAMA DE FLUJO: BTNGUARDARPROGRAMA_CLICK(OBJECT SENDER, EVENTARGS E)

btnCargarPrograma_Click(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void btnCargarPrograma_Click(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Lee el código fuente desde el archivo en disco y lo carga en el editor de texto, bloqueando la edición accidental.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: Botón de cargar; e: Argumentos del evento
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Instancia `Archivo<string>` y llama a `HacerModoLectura()`.
2. Limpia los cuadros de texto, tablas de errores y de símbolos.
3. Mediante un bucle `while` (!archivoTexto.FinArchivo), lee secuencialmente las líneas con `LeerObjeto()` y las añade al editor con `AppendText`.
4. Cierra el archivo con `CerrarArchivo()`.
5. Bloquea el editor temporalmente (`ReadOnly = true`, `Enabled = false`) para proteger el archivo recién cargado.
6. Llama a `InicializarArbolEnEspera()`.

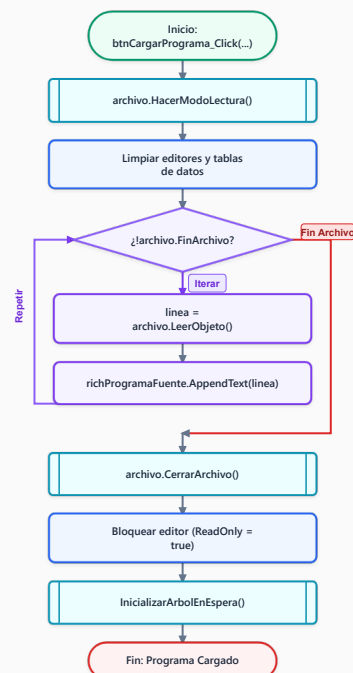


DIAGRAMA DE FLUJO: BTNCARGARPROGRAMA_CLICK(OBJECT SENDER, EVENTARGS E)

btnEditarPrograma_Click(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void btnEditarPrograma_Click(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Desbloquea el editor de código fuente permitiendo al usuario modificar manualmente el texto del programa.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: Botón de editar; e: Argumentos del evento
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Establece `richProgramaFuente.ReadOnly = false` y `richProgramaFuente.Enabled = true`. Notifica al usuario mediante un `MessageBox` indicando que el editor se encuentra nuevamente habilitado para escritura.

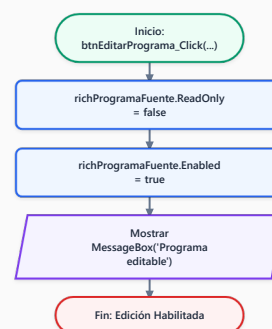


DIAGRAMA DE FLUJO: BTNEDITARPROGRAMA_CLICK(OBJECT SENDER, EVENTARGS E)

btnGuardarArchivo_Click(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void btnGuardarArchivo_Click(object sender, EventArgs e)
```

PROPÓSITO Y ROL EN EL SISTEMA

Guarda en disco la secuencia de tokens generados por el analizador léxico en el archivo secundario (Archivo2.txt).

PARÁMETROS Y RETORNO

Parámetros: sender: Botón de guardar tokens; e: Argumentos del evento

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Instancia 'Archivo<string>' con la ruta 'NombreArchivo2'.
2. Elimina el archivo previo si existe y abre el modo de escritura.
3. Escribe línea por línea los tokens contenidos en 'richArchivoDeTokens.Lines'.
4. Cierra el flujo de archivo y confirma la operación al usuario.

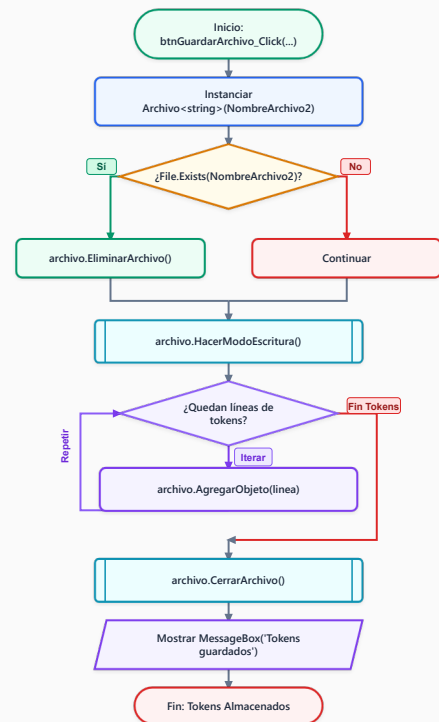


DIAGRAMA DE FLUJO: BTNGUARDARARCHIVO_CLICK(OBJECT SENDER, EVENTARGS E)

picLineas_Paint(...) y picLinea2_Paint(...)

PRIVATE

VOID

FORM1.CS

```
private void picLineas_Paint(object sender, PaintEventArgs e)
private void picLinea2_Paint(object sender, PaintEventArgs e)
```

PROPÓSITO Y ROL EN EL SISTEMA

Dibuja visualmente los números de línea correlativos alineados a la derecha en los márgenes de los editores de texto.

PARÁMETROS Y RETORNO

Parámetros: sender: PictureBox; e: Argumentos Paint con el objeto Graphics

Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Determina los índices del primer y último carácter visibles en el área cliente del RichTextBox.
2. Convierte esos índices en números de línea ('primeraLineaVisible' y 'ultimaLineaVisible').
3. Itera únicamente sobre las líneas que son visibles en la pantalla en ese momento.
4. Mide el ancho del texto del número con 'MeasureString' y lo dibuja con alineación derecha en el PictureBox usando una brocha color Teal ('Brushes.Teal').
5. Proporciona una experiencia de editor de código moderna y de alto rendimiento sin calcular líneas invisibles.

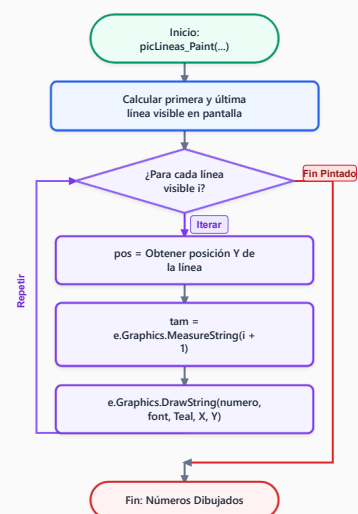


DIAGRAMA DE FLUJO: PICLINEAS_PAINT(...) Y
PICLINEA2_PAINT(...)

richArchivoDeTokens_TextChanged(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void richArchivoDeTokens_TextChanged(object sender, EventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

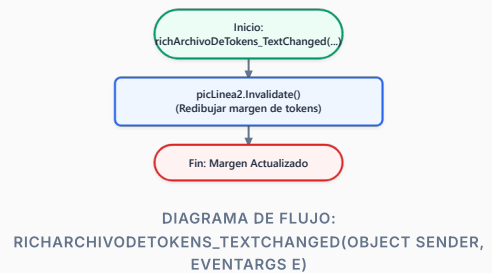
Fuerza el redibujado de la numeración de líneas del visor de tokens cuando su contenido cambia.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: Control richArchivoDeTokens; e: Argumentos del evento
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

Ejecuta `picLinea2.Invalidate()`, asegurando que la columna de numeración de tokens se sincronice visualmente de forma inmediata con los nuevos datos generados.



CompilarArbolJerarquia(AnalizadorSintactico? sintactico = null)

PUBLIC

VOID

FORM1.CS

```
public void CompilarArbolJerarquia(AnalizadorSintactico? sintactico = null)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

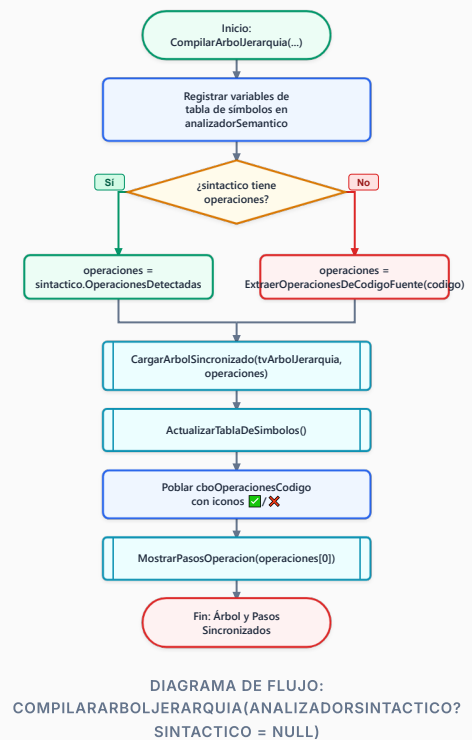
Construye y sincroniza visualmente el árbol de jerarquía, poblando el TreeView y el ComboBox de operaciones encontradas.

🔗 PARÁMETROS Y RETORNO

Parámetros: AnalizadorSintactico? sintactico: Instancia del analizador sintáctico con las operaciones detectadas
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Registra las variables existentes en la tabla de símbolos en `analizadorSemantico`.
2. Recupera las operaciones detectadas por el analizador sintáctico (`sintactico.OperacionesDetectadas`) o las extrae del código con `ExtraerOperacionesDeCodigoFuente`.
3. Invoca `analizadorSemantico.CargarArbolSincronizado(tvArbolJerarquia, operacionesDetectadas)` para renderizar los nodos visuales.
4. Llama a `ActualizarTablaDeSimbolos()` para reflejar los tipos y valores inferidos.
5. Pueba el control `cboOperacionesCodigo` con cada operación detectada, añadiendo indicadores de estado (✅ o ❌).
6. Muestra los pasos de la primera operación con `MostrarPasosOperacion(operacionesDetectadas[0])`.



ActualizarTablaDeSimbolos()

PRIVATE

VOID

FORM1.CS

```
private void ActualizarTablaDeSimbolos()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Sincroniza los identificadores de la lista en memoria con los tipos y valores del analizador semántico y refresca el DataGridView.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Itera sobre cada variable en 'ListaDeIdentificadoresOrdenada'.
2. Consulta 'analizadorSemantico.ObtenerTipo(nombre)' y actualiza 'TipoDeDato' (asignando 'int' por defecto si es nulo).
3. Consulta 'analizadorSemantico.ObtenerValor(nombre)' y actualiza el campo 'Valor' con el resultado calculado.
4. Limpia las filas de 'dgtTablaDeSimbolos' con 'Rows.Clear()'.
5. Inserta nuevamente cada identificador con su número, nombre, tipo de dato y valor actualizado.

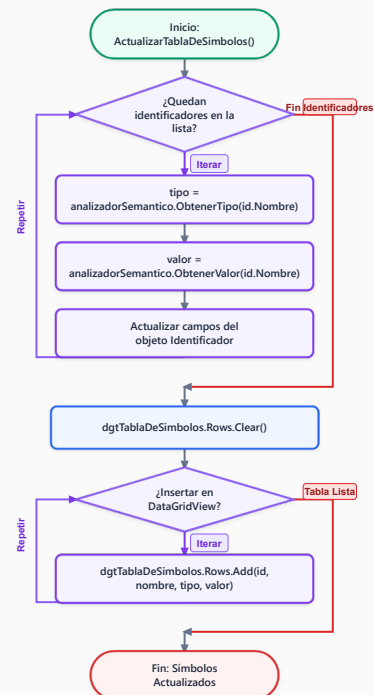


DIAGRAMA DE FLUJO: ACTUALIZARTABLADESIMBOLOS()

InicializarArbolEnEspera()

PRIVATE

VOID

FORM1.CS

```
private void InicializarArbolEnEspera()
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Restaura los controles de jerarquía y semántica a un estado visual neutro antes de ejecutar una compilación.

🔗 PARÁMETROS Y RETORNO

Parámetros: Ninguno

Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Limpia 'tvArbolJerarquia' y coloca un nodo informativo: '⚡ Presiona Analizar Todo para compilar el árbol' en color SlateGray.
2. Limpia y reinicia 'cboOperacionesCodigo' con el texto '(Presiona Analizar Todo)'.
3. Establece la etiqueta de estado 'lblEstadoSemantico' en 'Listo: presiona Analizar Todo'.
4. Limpia el cuadro de texto enriquecido de pasos jerárquicos 'rtbPasosJerarquia'.

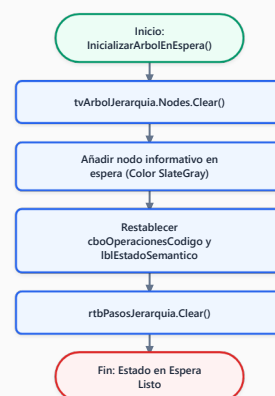


DIAGRAMA DE FLUJO: INICIALIZARARBOLENESPERA()

MostrarPasosOperacion(ResultadoSemanticoOperacion op)

PRIVATE

VOID

FORM1.CS

```
private void MostrarPasosOperacion(ResultadoSemanticoOperacion op)
```

PROPÓSITO Y ROL EN EL SISTEMA

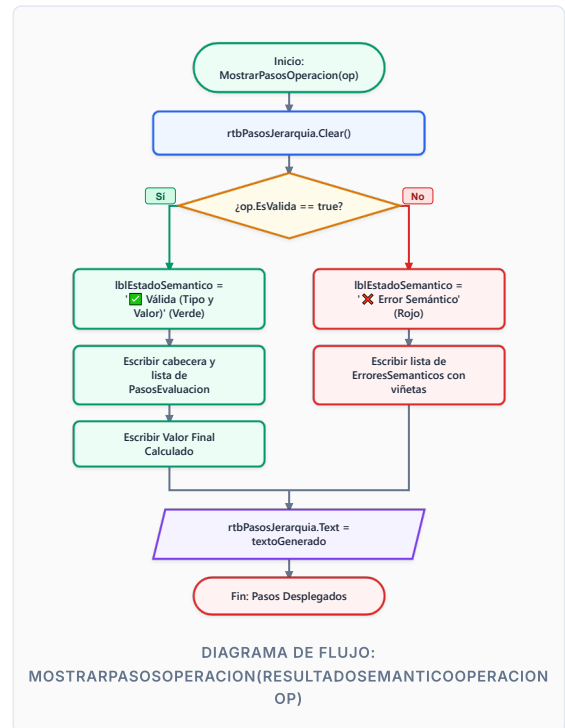
Formatea y despliega en el RichTextBox los pasos de evaluación por precedencia jerárquica y el resultado final calculado.

PARÁMETROS Y RETORNO

Parámetros: ResultadoSemanticoOperacion op: Operación semántica a exhibir
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Limpia 'rtbPasosJerarquia'.
2. Si 'op' es nula, retorna.
3. Si 'op.EsValida':
 - Fija 'lblEstadoSemantico' en verde: '✅ Jerarquía Válida' con el tipo y valor resultante.
 - Genera una cabecera con el texto de la operación original.
 - Itera y concatena cada paso de 'op.PasosEvaluacion'.
 - Añade el valor final calculado si está disponible.
4. Si '!op.EsValida':
 - Fija 'lblEstadoSemantico' en rojo: '❌ Error Semántico en la Operación'.
 - Concatena la lista de errores semánticos detallados ('op.ErroresSemanticos').
5. Asigna el texto formateado al control 'rtbPasosJerarquia'.



cboOperacionesCodigo_SelectedIndexChanged(object sender, EventArgs e)

PRIVATE

VOID

FORM1.CS

```
private void cboOperacionesCodigo_SelectedIndexChanged(object sender, EventArgs e)
```

PROPÓSITO Y ROL EN EL SISTEMA

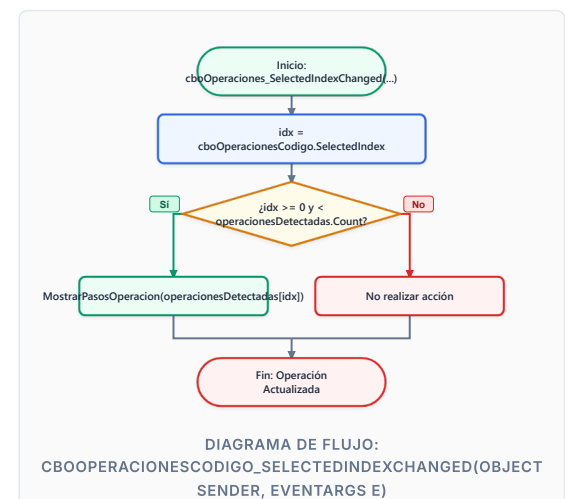
Actualiza la visualización de pasos cuando el usuario selecciona una operación diferente en la lista desplegable.

PARÁMETROS Y RETORNO

Parámetros: sender: Control ComboBox; e: Argumentos del evento
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Obtiene el índice de la opción seleccionada 'idx = cboOperacionesCodigo.SelectedIndex'.
2. Si el índice se encuentra dentro del rango válido de 'operacionesDetectadas', recupera la operación correspondiente.
3. Invoca a 'MostrarPasosOperacion(operacionesDetectadas[idx])' para refrescar los pasos en la interfaz.



```
private void tvArbolJerarquia_AfterSelect(object sender, TreeViewEventArgs e)
```

🔗 PROPÓSITO Y ROL EN EL SISTEMA

Sincroniza bidireccionalmente el nodo seleccionado en el árbol jerárquico con el ComboBox y los pasos detallados de resolución.

🔗 PARÁMETROS Y RETORNO

Parámetros: sender: TreeView; e: Argumentos con el nodo seleccionado
Retorno: void

⚙️ EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

1. Si `e.Node == null`, termina.
2. Recorre hacia arriba la jerarquía del nodo seleccionado (`actual = actual.Parent`) hasta encontrar un nodo que contenga en su propiedad `Tag` una instancia de `ResultadoSemanticoOperacion`.
3. Al encontrarlo, invoca a `MostrarPasosOperacion(opTag)`.
4. Identifica el índice de la operación y sincroniza el valor seleccionado en `cboOperacionesCodigo`, suspendiendo temporalmente el evento para no provocar llamadas recursivas.

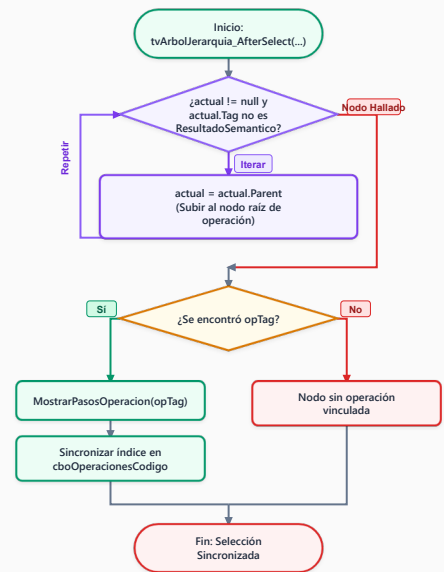


DIAGRAMA DE FLUJO:
TVARBOLJERARQUIA_AFTERSLECT(OBJECT SENDER,
TREEVIEWEVENTARGS E)

Módulo 9: Punto de Entrada de la Aplicación (Program.cs)

Punto de entrada primario (Main) del ensamblado ejecutable. Configura el subprocesamiento STAThread, habilita la compatibilidad con alta resolución de pantalla (High DPI) e inicia el bucle de mensajes de Windows Forms.

Main()

INTERNAL / STATIC

VOID

PROGRAM.CS

```
[STAThread]
static void Main()
```

PROPÓSITO Y ROL EN EL SISTEMA

Punto de entrada principal del ejecutable de la aplicación de escritorio.

PARÁMETROS Y RETORNO

Parámetros: Ninguno
Retorno: void

EXPLICACIÓN DETALLADA DEL FUNCIONAMIENTO

- Lleva el atributo `[STAThread]` que define el modelo de subprocesamiento de apartamento único requerido por Windows Forms para interactuar con el portapapeles y componentes COM.
- Llama a `ApplicationConfiguration.Initialize()` para configurar el renderizado de alta resolución (DPI awareness) y tipografías predeterminadas del sistema.
- Ejecuta el bucle de mensajes de Windows Forms llamando a `Application.Run(new Form1())`, manteniendo la ventana activa hasta que el usuario cierre la aplicación.

